14 haftalık yol haritasında 14. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 14

Yapılar, Dosyalar ve Bütünleştirici Proje: Gerçek Bir Program İnşa Etmek

struct ve typedef, iç içe yapılar ve yapı dizileri, -> ve const işaretçiler, enum, hizalama ve dolgu, FILE * ve akışlar, fopen kipleri, tamponlama, metin/CSV ve ikili dosyalar, Mini-OBS v1.0 ve final projesi

MODÜL VI · METİN, KAYIT VE KALICILIK · H13–H14

> HAFTANIN BÜYÜK SORUSU
>
> *Program kapandığında veriler nereye gider ve gerçek dünyadaki bir "kayıt" bellekte ve diskte nasıl temsil edilir?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
1.1 Öğrenme çıktıları ... 3
2. Geçen Haftadan: Karakter Dizilerinden Kayıtlara ... 3
3. Kavramsal Anlatım ... 4
3.1 Kayıt kavramı: paralel dizilerden struct'a ... 4
3.2 Atama, karşılaştırma, typedef ve iç içe yapılar ... 5
3.3 Yapı işaretçileri, -> ve const ... 6
3.4 enum ve union ... 6
3.5 Kaputun altında: hizalama ve dolgu ... 7
3.6 Dosyalar, akışlar ve fopen kipleri ... 7
3.7 Kaputun altında: tamponlama ... 8
3.8 Metin dosyası ve doğru okuma döngüsü ... 9
3.9 CSV: noktalı virgülle ayrılmış değerler ... 10
3.10 İkili dosya: fwrite, fread, fseek ve onaltılı döküm ... 11
4. Çözümlü Örnekler: Örnek Merdiveni ... 11
4.1 Örnek 2–6 (★–★★): typedef, enum, ilk dosya, iç içe yapı, yapı dizisi ... 11
4.2 Amiral örnek — Örnek 7 ★★: yapı işaretçisi ve -> ile güncelleme ... 14
4.3 Örnek 8–10 (★★–★★★): mini wc, dosya hataları, dolgu deneyi ... 16
4.4 Amiral örnek — Örnek 11 ★★★: CSV'den öğrenci listesinin yüklenmesi ... 17
4.5 Örnek 12 (★★★): ikili dosyaya yapı dizisi yazma ve okuma ... 19
5. Hata Avcısı ... 20
6. Mühendis Gözüyle: Veri Modelleme, Dayanıklılık ve Güvenlik ... 20
7. Mini-OBS v1.0: Bütünleştirme (Örnek 13) ... 21
7.1 Dosya biçimleri, testler ve kabul ... 22
8. Alıştırmalar, Atölye ve Final Projesi ... 24
8.1 Alıştırmalar ... 24
8.2 Atölye 1 — Final projesi atölyesi: kod incelemesi ve eşli hata ayıklama ... 26
8.3 Atölye 2 — Proje sunumları: canlı demo, mimari ve test senaryoları ... 26
8.4 Atölye 3 — Akran değerlendirmesi ve rubrik temelli geri bildirim ... 27
9. Kendini Değerlendirme ... 27
10. AP-2 Köprüsü ve Merak Kutusu ... 28
11. Özet, Kavram Haritası ve Terim Sözlüğü ... 28
12. Kaynaklar ... 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Program kapandığında veriler nereye gider ve gerçek dünyadaki bir "kayıt" bellekte ve diskte nasıl temsil edilir?

On üç haftadır yazdığımız her program aynı biçimde bitti: `return 0;` satırında değişkenler, diziler ve öbekteki bloklar yok oldu. Mini-OBS v0.13 öğrenci sayısı arttıkça `realloc` ile büyüyen bloklarda istediğimiz kadar öğrenciyi tutabiliyor, ama program kapanınca hepsini unutuyor ve bir öğrencinin verisi **altı ayrı paralel blokta** duruyor. Bu hafta iki eksik parçayı tamamlıyoruz: birbirine ait verileri tek bir **kayıt** tipinde toplayan **yapılar** (struct) ve veriyi program kapandıktan sonra da saklayan **dosyalar**. Sonra dönemin bütün kavramlarını Mini-OBS v1.0'da birleştirip final projesine geçiyoruz.

> GERÇEK DÜNYADAN · **Yılı iki haneyle saklamanın bedeli: 2000 Yılı Problemi (Y2K)**
>
> ABD Genel Muhasebe Ofisi'nin (GAO) 1997 tarihli bir tanıklık metnine göre bilgisayar sistemleri onlarca yıl boyunca, elektronik alandan tasarruf etmek ve işletim maliyetini düşürmek için yılı **iki haneyle** saklamıştı (1997 için "97"); bu biçimde 2000 ile 1900'ün ikisi de "00"dır. Metindeki örnekte 1930 doğumlu bir kişinin yaşı 2000'de −30 çıkar ve hak ettiği ödemeyi alamaz (Willemssen, 1997). Aynı hesabı `H14_K14_y2k_2038.c` ile yaptık: "1999'da yaş: 69", "2000'de yaş: -30".
>
> ABD Senatosu Y2K Özel Komitesi'nin 29 Şubat 2000 tarihli son raporu maliyetin yalnızca **tahmin** edilebildiğini vurgular: Ticaret Bakanlığı ABD kamu kurumları ve şirketleri için yaklaşık 100 milyar dolar, Yönetim ve Bütçe Ofisi federal hükümet için 8,34 milyar dolar tahmin etmiştir (United States Senate Special Committee on the Year 2000 Technology Problem, 2000). Ders: bir kaydın **alanlarını ve biçimini** seçmek, programın kendisinden uzun yaşayan bir karardır.

1.1 Öğrenme çıktıları

ÖÇ 14.1 struct ile ilişkili verileri tek bir kayıt tipinde modeller; typedef, iç içe yapılar ve yapı dizileri kullanır. *(Bölüm 3.1–3.4, Örnek 1–3, 5–6)*

ÖÇ 14.2 Yapıları fonksiyonlara değerle ve işaretçiyle aktarır; . ve -> operatörlerini doğru kullanır. *(Bölüm 3.3, Örnek 7, HA2)*

ÖÇ 14.3 Metin ve ikili dosyalarla kalıcı veri saklar (fopen/fclose, fprintf/fscanf, fgets/fputs, fwrite/fread) ve dosya hatalarını denetler. *(Bölüm 3.6–3.10, Örnek 4, 8–12, Bölüm 5)*

ÖÇ 14.4 Dönem boyunca edinilen kavramları bütünleştirerek modüler, menü güdümlü ve verisini kalıcı olarak saklayan bir uygulama geliştirir, test eder ve sunar. *(Bölüm 7–8; final projesi yönergesi)*

2. Geçen Haftadan: Karakter Dizilerinden Kayıtlara

> TAHMİN ET · **Geri çağırma: H13'ten üç soru (kodlar/quiz/)**
>
> - **S1 ('\0' ve boyut + 1).** `char ad[] = "MERT";` için `sizeof ad` ve `strlen(ad)` ne yazdırır? (`H14_Q01_sonlandirici.c`)
> - **S2 (fgets ve '\n').** Klavyeden `Ali` yazılıp Enter'a basılırsa program ne yazdırır? (`H14_Q02_fgets_satir_sonu.c`)
> - **S3 (== ile strcmp).** İki ayrı dizide aynı metin var. `p == q` ve `strcmp(p, q) == 0` ne verir? (`H14_Q03_strcmp.c`)

Yanıtlar (gerçek çıktılar). S1: "5 4": dizi, sonlandırıcıyı da tutar (boyut + 1). S2: "4 farklı": fgets satır sonunu da saklar; Örnek 11'de CSV satırlarının sonundaki '\n' ve Windows'tan gelen '\r' aynı nedenle silinir. S3: "0 1": `==` adresleri, `strcmp` içeriği karşılaştırır; iki yapı ise `==` ile hiç karşılaştırılamaz (HA1).

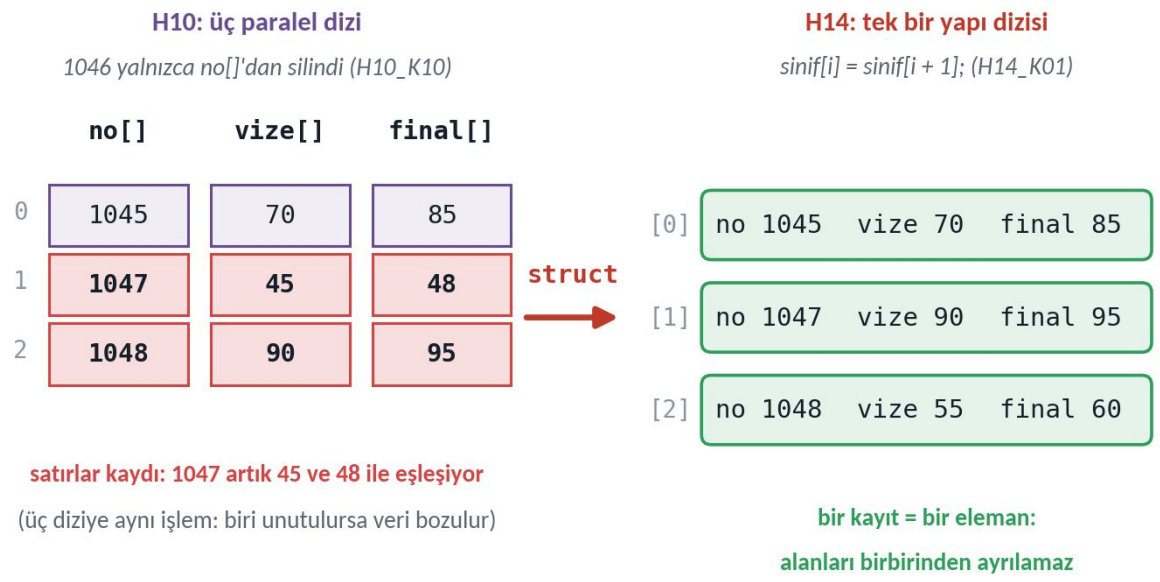
Önceki haftalardan açık kalanlar. H3'te 0x12345678 bellekte 78 56 34 12 sırasıyla durdu (`H03_K02_bayt_sirasi.c`): ikili dosyada da bu sırada durur (3.10). H7'nin menüsündeki sihirli 1, 2, 3 sayıları `enum MenuSecimi` olur (Örnek 3); H9'un çok dosyalı yapısı v1.0'da dört modüle büyür (Bölüm 7). H10'da bir dizi unutulunca kayıtlar kaydı (`H10_K10_paralel_diziler.c`): yapıda bir kayıt tek elemandır (3.1). H12'nin `realloc`'u yapı dizisini büyütür, baytlarını `fwrite` diske yazar; H13'ün durum makinesi ve strtok/strtol'u mini wc'ye (Örnek 8) ve CSV ayrıştırmaya (3.9, Örnek 11) dönüşür.

3. Kavramsal Anlatım

3.1 Kayıt kavramı: paralel dizilerden struct'a

Bir öğrencinin numarası, adı ve notları birbirine **ait** verilerdir: silerken, taşırken ya da sıralarken birlikte hareket etmelidir. H10'da onları aynı indisli **paralel dizilerde** tuttuk; `H10_K10_paralel_diziler.c`'de 1046 silinirken yalnızca `no[]` kaydırıldı, `vize[]` ve `final[]` unutuldu ve satırlar kaydı (Şekil 14.1, solda). v0.13'te paralel blok sayısı altıya çıktı: her silme ya da büyütme altı kez yazılmalıdır.

Şekil 14.1
Paralel dizilerden yapı dizisine: H10_K10'un hatalı silmesi (solda) ve aynı silmenin yapı dizisinde tek atamayla yapılması (sağda; H14_K01'in gerçek çıktısı)



Yapı (structure, `struct`) farklı tiplerden **üyeleri** (member) tek bir ad altında toplayan bir tiptir; bir yapı nesnesi bir **kayıttır** (record). `H14_K01_paralel_struct.c`'de `sinif[i] = sinif[i + 1];` bir kaydın bütün alanlarını **birlikte** taşır ("no 1047 vize 90 final 95"; Kernighan & Ritchie, 1988, Bölüm 6; King, 2008, Bölüm 16). Yapı tanımı bir **tip** bildirir, bellek ayırmaz: `struct Nokta` sözcük çiftinin tamamı tipin adıdır (`Nokta` **etiket**tir). Üyelere **nokta operatörüyle** erişilir (`a.x`). İlk değerler üyelere bildirim sırasıyla gider; C99'dan beri **belirlenmiş ilk değerle** (designated initializer) üyenin adı da yazılabilir; adı geçmeyen üyeler sıfırlanır (ISO/IEC, 2018, 6.7.9). Örnek 1:

H14_01_nokta_uzaklik.c · satır 12–22 gcc … -lm

```c
struct Nokta {                  /* yeni bir TİP: iki double'lık kayıt */
    double x;
    double y;
};

double uzaklik(struct Nokta p, struct Nokta q)
{
    double dx = q.x - p.x;
    double dy = q.y - p.y;
    return sqrt(dx * dx + dy * dy);
}
```

Gerçek çıktı: “A(1.0, 2.0) B(4.0, 6.0)”, “|AB| = 5.00”, “sizeof(struct Nokta) = 16 bayt”. `uzaklik(a, b)` iki yapıyı **değerle** alır: `p` ve `q`, a ile b’nin 16’şar baytlık kopyalarıdır (H8). Tanımın sonundaki `;` zorunludur; unutulursa derleyici anlaşılması güç bir hatayı bir sonraki satırda verir.

3.2 Atama, karşılaştırma, typedef ve iç içe yapılar

Atama kopyalar, diziler dahil. `H14_K02_atama_kopya.c`’de `Oyuncu b = a;` bütün üyeleri kopyalar; `b.ad` a’dan **ayrı** bir dizidir: `b.ad[0] = 'X'; b.puan = 10;` sonrasında “a.ad = CAN, b.ad = XAN”, “a.puan = 70, b.puan = 10”. Oysa iki dizi birbirine atanamaz (`H14_K03_dizi_atama_hatasi.c`: “assignment to expression with array type”). Fonksiyonlar için de aynı kural geçerlidir: dizi döndüren fonksiyon yazılamaz, yapı döndüren yazılır (`Nokta orta_nokta(Nokta p, Nokta q)`: “orta nokta = (2.0, 3.0)”).

Karşılaştırma alan alan yapılır. `==` yalnızca aritmetik tiplerde ve işaretçilerde tanımlıdır (ISO/IEC, 2018, 6.5.9); `a == b` derleme hatasıdır (HA1). Baytları `memcmp` ile karşılaştırmak da güvenilmezdir, çünkü yapının içinde değeri **belirsiz** dolgu baytları olabilir (6.2.6.1; 3.5). `H14_K04_karsilastirma.c`’de alanları aynı iki kaydın dolguları farklı doldurulunca sonuç “alan alan: EŞİT, memcmp: FARKLI” oldu. SEI CERT C bunu kural olarak koyar: dolgu verisini karşılaştırmayın (EXP42-C; Carnegie Mellon University Software Engineering Institute, 2016).

H14_02_ogrenci_typedef.c · satır 16–21

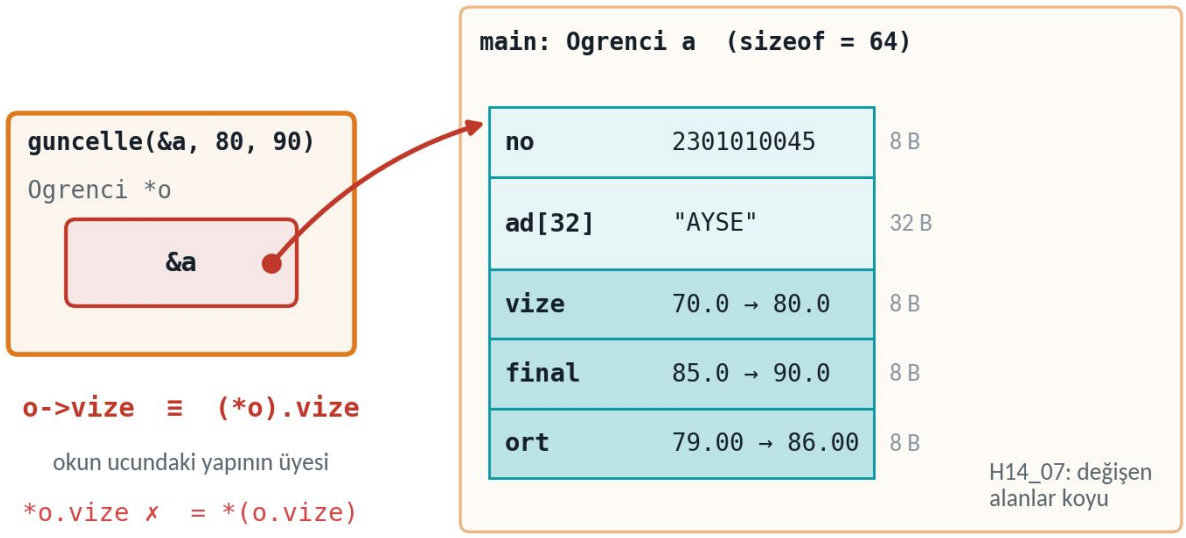
```c
typedef struct Ogrenci {
    long long no;
    char ad[AD_BOYUT];
    double vize;
    double final;
} Ogrenci;
```

`typedef` bir tipe **yeni bir ad** verir, yeni tip oluşturmaz: bu tanımdan sonra `struct Ogrenci` ile `Ogrenci` aynı tiptir. Örnek 2’nin `Ogrenci ogrenci_oku(void)` fonksiyonu yapıyı **değer olarak döndürür**: `Ogrenci a = ogrenci_oku();` bütün üyeleri a’ya kopyalar. Bir yapının üyesi başka bir yapı olabilir: Örnek 5’te `Ogrenci` iki `Tarih` üyesi taşır ve erişim zincirlenir: `o.dogum.yil` (“sizeof(Tarih) = 12, sizeof(Ogrenci) = 64”). Yapılardan dizi de kurulur: `Ogrenci sinif[4]`; `sinif[i].ad` i. kaydın adıdır ve H10’un desenleri (en büyüğün indisi, doğrusal tarama, kaydırarak silme) yapı dizisinde aynen çalışır (Örnek 6).

3.3 Yapı işaretçileri, -> ve const

Bir fonksiyonun bir kaydı **değiştirmesi** gerekiyorsa ona yapının **adresi** verilir (H12'deki takas gibi). Yapı işaretçisiyle üyeye erişmek için **ok operatörü** vardır: `o->vize`, `(*o).vize` demektir (Kernighan & Ritchie, 1988, Bölüm 6.2). Parantez şarttır: `.` operatörü `*`'dan önce bağlanır, `*o.vize` ifadesi `*(o.vize)` olarak okunur ve derlenmez (HA2). Kural kısadır: **solda yapı varsa nokta, işaretçi varsa ok** (Şekil 14.2).

Şekil 14.2
Yapı işaretçisi için kutu–ok diyagramı (Örnek 7): o, main'deki a'yı gösterir; o->vize okun ucundaki yapının vize üyesidir



Yapıyı yalnızca **okuyan** fonksiyonlar `const Ogrenci *` alır: kopya yapılmaz (8 baytlık adres gider) ve derleyici yazmaya izin vermez. Değerle aktarımın bedeli kopyadır: Örnek 7'de 64 baytlık yapı her çağrıda kopyalanır. `H14_K05_kopya_maliyeti.c` 4096 baytlık bir yapıyla bir milyon çağrıyı ölçtü: değerle 0,034 s, işaretçiyle 0,003 s (bu makinede tek bir çalıştırma; yorumlanacak olan orandır). Mini-OBS v1.0 bu yüzden raporlarda `const Sinif *` ve `const Ogrenci *` kullanır.

3.4 enum ve union

H14_03_enum_menu.c · satır 13–19

```
13  typedef enum {
14      MENU_HARF = 1,          /* 1: ortalamadan harf notu */
15      MENU_TABLO,             /* 2: harf tablosu          */
16      MENU_CIKIS              /* 3                        */
17  } MenuSecimi;
18
19  typedef enum { AA, BA, BB, CB, CC, DC, DD, FD, FF } HarfNotu;
```

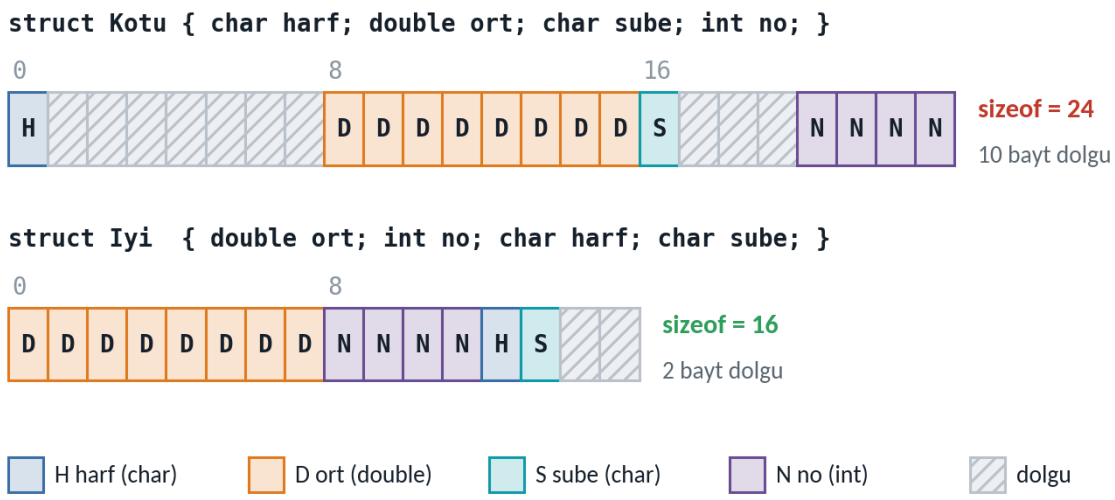
Numaralandırma (enumeration, `enum`) adlandırılmış tamsayı sabitleri tanımlar. Sabitlerin tipi `int`'tir; ilk sabit belirtilmezse 0'dır, sonrakiler birer artar: `MENU_TABLO` 2, `MENU_CIKIS` 3; `HarfNotu`'nda AA 0, FF 8'dir (ISO/IEC, 2018, 6.7.2.2). Böylece `case 3:` yerine `case MENU_CIKIS:` yazılır ve sabit bir dizide indis olur (Örnek 3: "AA=0 BA=1 BB=2 CB=3 CC=4 DC=5 DD=6 FD=7 FF=8"). Derleyici bir enum değişkenine aralık dışı bir sayının atanmasını engellemez; `switch`'teki `default` dalı bu yüzden gereklidir.

> MERAK KUTUSU · **union: aynı baytlar, farklı yorumlar**
>
> `union`'da bütün üyeler **aynı adresten** başlar; boyut en büyük üyeye yeter (`H14_K06_union.c`: "sizeof(Deger) = 8") ve aynı anda en çok bir üyenin değeri saklanır (6.7.2.1). Son yazılmayan üye okunursa baytlar yeni tipte **yeniden yorumlanır** (6.5.2.3, dipnot): `d.f = 1.0f`'tan sonra `d.i` "0x3F800000" verdi (H3'teki IEEE 754). Güvenli kullanım **etiketli union**'dır: geçerli üyeyi bir `enum` alanı tutar ("tamsayı 42").

3.5 Kaputun altında: hizalama ve dolgu

İşlemci belleği sözcükler hâlinde okur; bu yüzden her tipin bir **hizalama** (alignment) gereksinimi vardır: nesnenin adresi bu sayının katı olmalıdır (x86-64 Linux'ta `_Alignof`: char 1, int 4, double 8). Derleyici üyeleri **bildirim sırasıyla** yerleştirir; her üyeyi hizalı bir adrese koymak için araya, dizide bir sonraki elemanın da hizalı olması için sona **dolgu** (padding) baytları ekler; yapının başında dolgu olamaz (ISO/IEC, 2018, 6.7.2.1; Bryant & O'Hallaron, 2016, Bölüm 3.9.3). `offsetof(tip, üye)` (`stddef.h`) üyenin yapının başından uzaklığını verir.

Şekil 14.3
Aynı dört üye, iki sıra: bayt haritaları H14_10_dolgu_deneyi.c'nin gerçek çıktısından (x86-64 Linux, GCC)



Örnek 10'un ölçümü: üyelerin toplamı 14 bayt; `struct Kotu` (char, double, char, int) 24 bayt (10 bayt dolgu), aynı üyeler büyükten küçüğe dizilince `struct Iyi` 16 bayt (2 bayt dolgu); 1000 kayıtlık bir dizide fark 8 000 bayttır. **Kural:** üyeleri hizalamaya göre büyükten küçüğe dizmek dolguyu çoğu zaman en aza indirir. Bu değerler **ABI'ye** (burada x86-64 System V) bağlıdır; başka bir sistemde farklı olabilir. Mini-OBS'nin `Ogrenci` yapısında hiç dolgu yoktur: 8 + 32 + 32 + 3 × 24 = 144 bayt (`H14_K16_obs_yerlesim.c`).

3.6 Dosyalar, akışlar ve fopen kipleri

Dosya, kalıcı bir ortamda duran ve adı olan bir bayt dizisidir. C dosyayla bir **akış** (stream) üzerinden konuşur: `fopen` dosyayı açar ve bir `FILE *` döndürür; `FILE` nesnesi konumu, hata ve dosya sonu göstergelerini ve bir **tampon** tutar (Kernighan & Ritchie, 1988, Bölüm 7.5; King, 2008, Bölüm 22). Her program açık üç akışla başlar: `stdin`, `stdout` ve hata iletileri için `stderr`; `printf(…)`, `fprintf(stdout, …)` demektir. Kabukta `>` stdout'u, `2>` stderr'i dosyaya yönlendirir. Standarda göre stderr tam tamponlu değildir; stdout etkileşimli bir aygıta bağlı değilse tam tamponludur (ISO/IEC, 2018, 7.21.3): `H14_K08_akislar.c` iki akışı aynı dosyaya yönlendirince sıra "2. satır (stderr)", "4. satır (stderr)", "1. satır (stdout)", "3. satır (stdout)" oldu.

`fopen(ad, kip)` dosyayı belirtilen **kipte** açar (Tablo 14.1); `b` eklenince (`rb`, `wb`, `r+b`) **ikili** kip olur. C11 ile gelen `x` dosya zaten varsa açmayı başarısız kılar ve yanlışlıkla üzerine yazmayı önler (ISO/IEC, 2018, 7.21.5.3; Microsoft, 2025). Son sütun `H14_K07_dosya_kipleri.c`'nin aynı dosyada sırayla yaptığı gerçek denemelerdir ('|' satır sonu).

Tablo 14.1
fopen kipleri ve H14_K07'nin gerçek denemeleri

Kip	İşlem	Dosya yoksa	Dosya varsa	H14_K07 (gerçek)
`r`	okur	**başarısız** (NULL)	baştan okur	açılamadı (No such file or directory)
`w`	yazar	oluşturur	**içeriği siler**	"ilk|"; sonra "yeni|"
`a`	sona ekler	oluşturur	korur, sona yazar	"ilk|ek|"
`r+`	okur ve yazar	**başarısız**	korur, baştan üzerine yazar	"XY" yazıldı: "XYk|ek|"
`w+` / `a+`	okur ve yazar / sona ekler	oluşturur	siler / korur	"" / "yeni|son|"
`wx` (C11)	yazar	oluşturur	**başarısız**	açılamadı (File exists); yoksa "tek|"

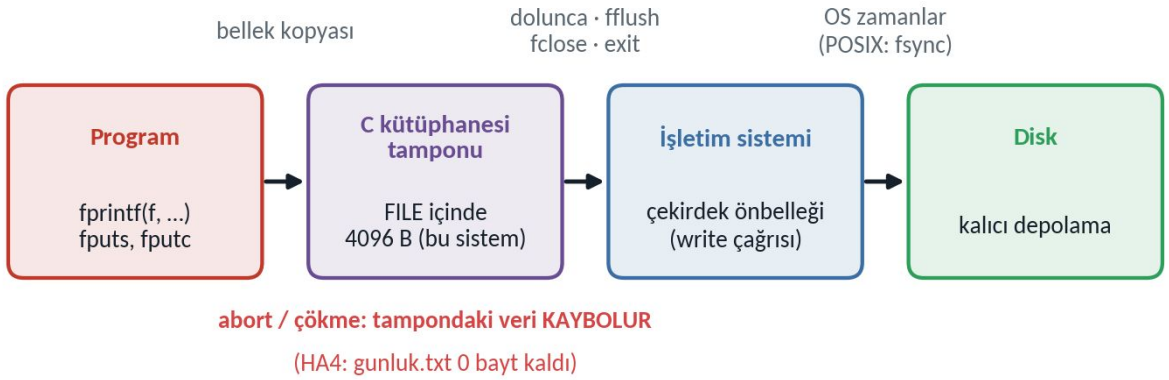
fopen başarısız olabilir ve o zaman NULL döndürür; NULL bir akışla `fprintf` çağırmak tanımsız davranıştır (HA3'te program çöktü). Her `fopen` sonucu denetlenir, neden `perror` ile yazılır: `perror(s)` stderr'e "s: ileti" yazar; ileti `errno`'ya karşılık gelen metindir (7.21.10.4). fopen'ın başarısızlıkta `errno`'yu ayarlaması **POSIX'te ve Windows CRT'de** güvencelidir; C standardı bunu zorunlu tutmaz. Örnek 9 sıradan bir kullanıcı (nobody) olarak dört dosyayı denedi: "olmayan.txt: No such file or directory", "gizli.txt: Permission denied", "klasor: Is a directory", "4 dosyadan 3 tanesi okunamadı.". **fclose da başarısız olabilir:** önce tampondaki veriyi yazar, sonra kapatır; başarıda 0, hatada `EOF` döndürür (7.21.5.1). Disk dolduysa hata ancak bu sırada ortaya çıkar; yazılan dosyada `fclose`'un dönüşü bu yüzden denetlenir.

3.7 Kaputun altında: tamponlama

`fprintf` veriyi doğrudan diske yazmaz: veri önce C kütüphanesinin **tamponuna**, oradan bir sistem çağrısıyla (Linux'ta `write`) işletim sistemine, oradan da işletim sisteminin uygun gördüğü bir anda diske gider (Şekil 14.4; Bryant & O'Hallaron, 2016, Bölüm 10). Tamponlama binlerce küçük yazımı az sayıda sistem çağrısında toplar.

`H14_K09_tampon_gozlem.c` dosyanın işletim sistemindeki boyutunu ikinci bir akışla ölçtü: 17 karakterlik `fprintf`'ten sonra 0 bayt, `fflush`'tan sonra 17 bayt; karakter karakter yazmada 4 097. karakterde tampon kendiliğinden boşaldı (4 113 bayt): tampon 4 096 baytti, dosya sisteminin blok boyutu da 4 096. `BUFSIZ` (burada 8 192) ise standarda göre `setbuf`'un kullandığı tamponun boyutudur (ISO/IEC, 2018, 7.21.1); kütüphanenin akışlara kendiliğinden verdiği tampon bundan farklı olabilir.

Şekil 14.4
Program → C kütüphanesi tamponu → işletim sistemi → disk; alt şerit H14_K09'un gerçek ölçümü



H14_K09'un ölçümü (ikinci bir akışla dosyanın işletim sistemindeki boyutu):



> KAPUTUN ALTINDA · **Tampondaki veri ne zaman kaybolur?**
>
> `fflush(f)` tamponu işletim sistemine iletir; `fclose(f)` iletir ve kapatır. Program **normal** biterse (`main`'den dönüş ya da `exit`) açık akışların hepsi boşaltılır ve kapatılır (ISO/IEC, 2018, 7.22.4.4). **Anormal** bitişte (çökme, `abort`, süreç öldürülmesi) tampondaki veri kaybolabilir: `abort`'ta akışların boşaltılıp boşaltılmayacağı uygulamaya bağlıdır (7.22.4.1). HA4'te üç kayıt yazıldı, akış kapatılmadı, program `abort()` ile bitti: dosya **0 bayt** kaldı. `fflush` veriyi yalnızca işletim sistemine teslim eder; elektrik kesintisine karşı POSIX'te ayrıca `fsync` gerekir.

3.8 Metin dosyası ve doğru okuma döngüsü

Tablo 14.2
Metin dosyası fonksiyonları (stdio.h) ve dönüş değerleri

Fonksiyon	Ne yapar?	Dönüş değeri
`fprintf(f, biçim, …)`	printf gibi biçimlendirip f'ye yazar	yazılan karakter sayısı; hatada negatif
`fputs(s, f)` / `fputc(c, f)`	dizgiyi / karakteri yazar ('\n' eklemez)	negatif olmayan değer / c; hatada EOF
`fgets(s, n, f)`	en çok n − 1 karakter ya da satır sonuna dek okur, '\n'i saklar	s; hiç karakter okunamadan dosya biterse ya da hatada NULL
`fgetc(f)`	bir sonraki baytı okur	`int`: bayt (0–255) ya da EOF
`fscanf(f, biçim, …)`	scanf gibi okur	atanan alan sayısı; hiçbiri atanmadan girdi biterse EOF

Döngü koşulu her zaman okuma fonksiyonunun dönüş değeridir: `while (fgets(satir, sizeof satir, f) != NULL)`. `while (!feof(f))` yaygın bir karşıt örüntüdür: `feof` dosya sonunu önceden haber vermez, yalnızca bir okuma dosya sonuna **çarptıktan sonra** doğru olur. Son satır okunduğunda `feof` hâlâ yanlıştır; döngü bir tur daha döner, `fgets` NULL döndürür ve tampondaki son

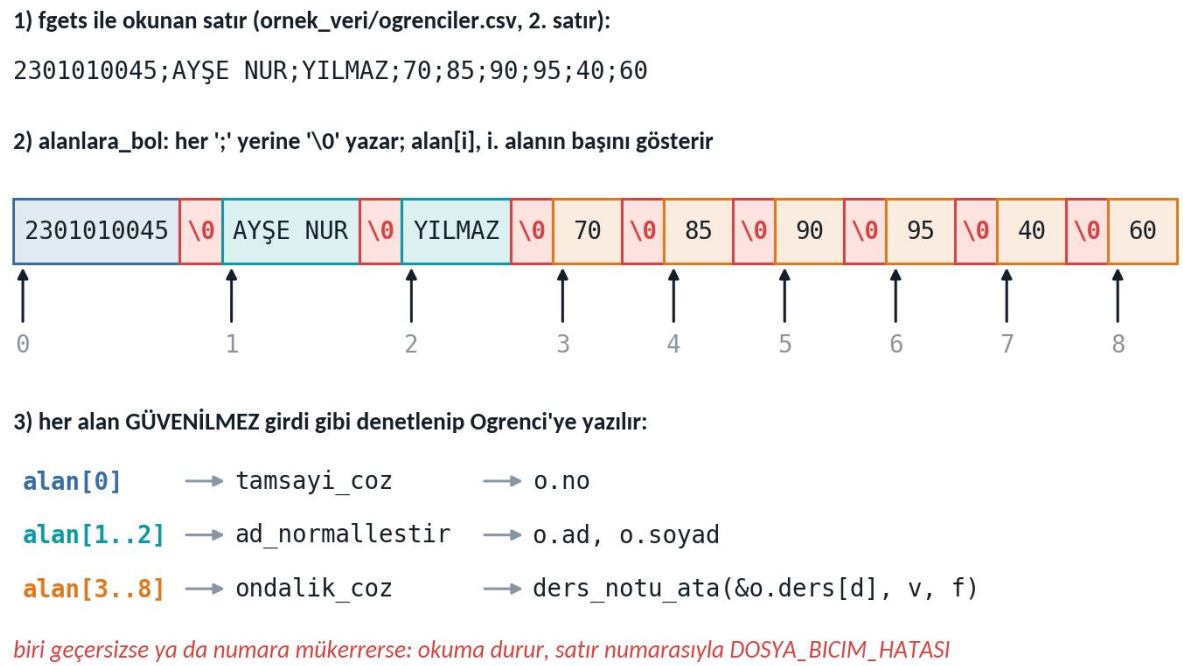
satır ikinci kez işlenir: HA5 üç kayıtlık dosyada “4 kayıt, toplam 335” yazdı, düzeltilmiş sürüm “3 kayıt, toplam 245”.

`fscanf` için koşul “beklenen alan sayısına eşit mi?” olmalıdır: `while ((r = fscanf(f, "%lld %lf %lf", &no, &vize, &final)) == 3)`. Döngüden sonra `r` nedeni söyler: EOF ise dosya bitti, 0–2 ise bir kayıt bozuk (`H14_K10_fscanf_donus.c`: “saglam.txt: 2 kayıt okundu; dosya sonu (fscanf EOF döndürdü)”, “bozuk.txt: 2 kayıt okundu; 3. kayıtta fscanf 1 döndürdü: bozuk veri”). `!= EOF` yazılsaydı bozuk veride fscanf hep 0 döndürür ve döngü bitmezdi. “Dosya sonu mu, okuma hatası mı?” sorusunu döngüden sonra `feof` ve `ferror` yanıtlar.

3.9 CSV: noktalı virgülle ayrılmış değerler

CSV (comma-separated values) her kaydın bir satır, her alanın bir ayırıcıyla ayrıldığı metin biçimidir; tablolama programları onu doğrudan açar. Ondalık ayırıcının virgül olduğu Türkçe bölge ayarlarında liste ayırıcısı olarak genellikle noktalı virgül kullanılır; Mini-OBS de `;` ve ondalıkta nokta kullanır. Bir satırı okumak üç adımdır: `fgets` ile satır, ayırıcı noktalarında bölme, her alanı **denetleyerek** çözümleme (Şekil 14.5).

Şekil 14.5
Bir CSV satırının alanlara ayrıştırılması (Mini-OBS v1.0, dosya.c): ';' yerine '\0', alan[] başlangıç adresleri, denetimli çözümleme



H13’ün `strtok`’u akla ilk gelen araçtır (Örnek 11 onu kullanır), ama **ardışık ayırıcıları tek sayar** ve boş alanı yutar. `H14_K11_strtok_bos_alan.c` aynı satırı iki yolla böldü: “strtok : 3 alan: [1] [ÇELİK] [70]”; “alanlara_bol: 4 alan: [1] [] [ÇELİK] [70]”. strtok’la ad alanı kayboldu, soyad ad yerine geçti; alan sayısı denetlenmeseydi kayıt sessizce bozulurdu. Mini-OBS’nin `alanlara_bol`’u her `;` noktasında böler ve boş alanı da sayar. İkisi de satırı **değiştirir** (`;` → `'\0'`). Dosyadan gelen her alan klavye girdisi kadar güvenilmezdir: sayılar `strtoll`/`strtod` ve “metnin tamamı sayı mı?” denetimiyle çözülür (H13).

3.10 İkili dosya: fwrite, fread, fseek ve onaltılı döküm

Metin dosyası sayıları **karakter** olarak yazar; **ikili** (binary) dosya nesnenin bellekteki **baytlarını** olduğu gibi yazar. `fwrite(p, boyut, adet, f)` p'den başlayan `adet` tane `boyut` baytlık öğeyi yazar ve yazdığı öğe sayısını döndürür; `fread` **okuyabildiği** öğe sayısını döndürür: dosya sonunda ya da hatada istenenden az olabilir (7.21.8). Her kayıt aynı boyutta olduğundan k. kayda doğrudan gidilir: `fseek(f, k * sizeof(Kayit), SEEK_SET)`; `ftell(f)` ikili akışta konumu bayt olarak verir (metin akışında değeri yalnızca fseek'e geri verilmek içindir, 7.21.9.4).

Linux'ta `od -A x -t x1z dosya` (ya da `xxd`, `hexdump -C`), Windows'ta PowerShell'in `Format-Hex dosya` komutu baytları onaltılı yazar. `H14_K12_sayi_metin_ikili.c` H3'teki 0x12345678'i iki dosyaya yazdı ("sayi.txt: 10 karakter, sayi.bin: 4 bayt"): sayi.txt'nin dökümü `33 30 35 34 31 39 38 39 36 0a` (rakamların ASCII kodları), sayi.bin'inki `78 56 34 12`: H12'de `H12_11_bayt_inceleme.c`'yle gördüğümüz küçük-uçlu sıra, bu kez diskte.

GERÇEK ÇIKTI: ÖRNEK 12'NİN İKİLİ DOSYASI (3 × 24 BAYT)

```
$ od -A x -t x1z kayitlar.dat
000000 01 00 00 00 45 4d 52 45 00 00 00 00 00 00 00 00  >....EMRE........<
000010 00 00 00 00 00 c0 53 40 02 00 00 00 43 41 4e 00  >......S@....CAN.<
000020 00 00 00 00 00 00 00 00 66 66 66 66 66 16 56 40  >........fffff.V@<
000030 03 00 00 00 5a 45 59 4e 45 50 00 00 00 00 00 00  >....ZEYNEP......<
000040 00 00 00 00 00 00 4a 40                          >......J@<
000048
```

Örnek 12'nin `Kayit`'i `{ int no; char ad[8]; double ort; }` 24 bayttır. İlk satırda `01 00 00 00` no = 1'dir (küçük-uçlu `int`), `45 4d 52 45 00 00 00 00` "EMRE" ve sonlandırıcılar, ardından gelen 4 sıfır bayt **dolgudur**: `ort` 8'in katı olan 16. bayttan başlar (`00 00 00 00 00 c0 53 40` = 79.0). Dolgu dosyaya da yazılır (72 bayt; aynı veri metin olarak 40 bayt). İkili dosya hızlı ve kayıpsızdır (double'ın 8 baytı aynen döner), ama **taşınabilir değildir**: dolgu, bayt sırası, `sizeof(int)` ve kayan nokta biçimi derleyiciye ve makineye bağlıdır. Dolgu baytlarına bellekteki eski değerler düşebilir; bunların dosyaya sızmaması için kayıtlar yazılmadan önce sıfırlanır (`memset`; SEI CERT C DCL39-C).

> NOT • **Windows'ta metin ve ikili kip**
>
> Linux'ta metin ve ikili kip aynıdır. Windows'ta (Microsoft C çalışma zamanı) **metin kipinde** yazarken her '\n' diske "\r\n" (0D 0A) olarak gider, okurken tek '\n'e çevrilir; ayrıca metin kipinde okurken Ctrl+Z (0x1A) dosya sonu sayılır (Microsoft, 2025). Bu dönüşümler ikili veriyi bozar: bir `double`'ın içinde 0A ya da 1A baytı geçebilir. İkili veri **her zaman** `rb`/`wb` ile açılır. Windows'ta yazılmış bir CSV Linux'ta okunurken satır sonundaki '\r' de temizlenir (Mini-OBS'nin `csv_oku`'su bunu yapar).

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H14_01_…` – `H14_12_…`; 13. basamak Mini-OBS v1.0, Bölüm 7). Her biri bir yeni öge ekler; hepsi `gcc -std=c17 -Wall -Wextra -pedantic` ile **sıfır uyarıyla** derlenir, ASan + UBSan altında temiz çalışır ve çıktılar gerçektir. Örnek 1 Bölüm 3.1'dedir; **Örnek 7** ve **Örnek 11** tam şablonla işlenir. Kesitlerin satır numaraları dosyalarla aynıdır.

4.1 Örnek 2–6 (★–★★): typedef, enum, ilk dosya, iç içe yapı, yapı dizisi

İlk beş basamak 3.1–3.8'deki kavramları birer küçük programda kullanır; kesitler dosyalardan, çıktılar gerçek çalıştırmalardandır.

Örnek 2 ★ — typedef ve yapı döndüren fonksiyon (`H14_02_ogrenci_typedef.c`). Yapı tanımı 3.2'dedir. `ogrenci_oku` yerel bir `Ogrenci` doldurup **değer olarak döndürür**; hata "geçersiz kayıt" işaretiyle (`no = 0`) taşınır:

H14_02_ogrenci_typedef.c · satır 25–33

```
25  Ogrenci ogrenci_oku(void)
26  {
27      Ogrenci o = {0};                    /* bütün üyeler sıfır */
28      printf("No, ad, vize, final: ");
29      if (scanf("%lld %31s %lf %lf", &o.no, o.ad, &o.vize, &o.final) != 4) {
30          o.no = 0;                       /* geçersiz kayıt işareti */
31      }
32      return o;
33  }
```

Gerçek çıktı: "2301010045 Ayşe vize 70.0 final 85.0 ort 79.0", "sizeof(Ogrenci) = 56 bayt". `ogrenci_yazdir(Ogrenci o)` kaydın kopyasını alır. Örnek 7'de daha iyi yolu göreceğiz: kaydı işaretçiyle doldurup dönüş değerini başarı için kullanmak.

Örnek 3 ★ — enum ile menü (`H14_03_enum_menu.c`). `ALT_SINIR[h]` dizisi enum değerini indis olarak kullanır; H5'teki dokuz dallı `if` merdiveni bir döngüye dönüşür:

H14_03_enum_menu.c · satır 26–34

```
26  HarfNotu harf_bul(int ortalama)
27  {
28      for (HarfNotu h = AA; h < FF; h++) {    /* enum da bir tamsayıdır */
29          if (ortalama >= ALT_SINIR[h]) {
30              return h;
31          }
32      }
33      return FF;
34  }
```

Gerçek çıktı: "78 -> BB (enum değeri 2)". Menü `do-while` + `switch ((MenuSecimi)secim)` ile (H7) `case MENU_HARF:` … `case MENU_CIKIS:` adlarıyla kurulur; geçersiz seçim `default` dalına düşer ("Geçersiz seçim: 7").

Örnek 4 ★ — ilk dosya (`H14_04_metin_dosyasi.c`). Program üç satırı `"w"` kipiyle yazar ve `fclose`'un sonucunu denetler; sonra dosyayı `"r"` ile yeniden açıp okur:

H14_04_metin_dosyasi.c · satır 32–43

```
32      f = fopen(DOSYA, "r");                  /* okumak için */
33      if (f == NULL) {
34          perror(DOSYA);
35          return 1;
36      }
37      char satir[128];
38      int sayac = 0;
39      while (fgets(satir, sizeof satir, f) != NULL) {
40          sayac++;
41          printf("%d: %s", sayac, satir);     /* satir '\n' içerir */
42      }
43      fclose(f);
```

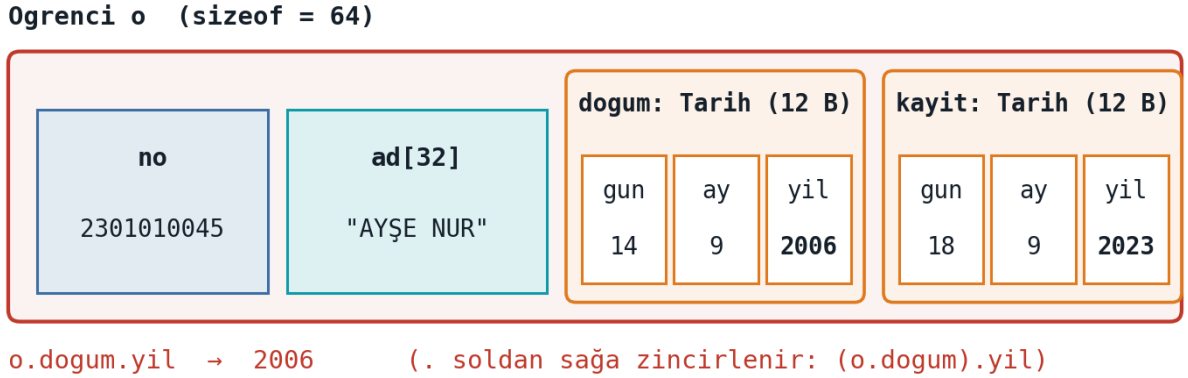
Gerçek çıktı: “1: 2301010045 70 85” … “3 satır okundu.”. Okunan her satır '\n'i de içerdiği için `printf("%d: %s", …)` yeni satır eklemez. **Değiştir:** "w" yerine "a" yazıp programı iki kez çalıştırın; dosya kaç satır olur?

Örnek 5 ★★ — iç içe yapı (`H14_05_ic_ice_tarih.c`). `Ogrenci`'nin içinde iki `Tarih` yapısı vardır (Şekil 14.6); iç içe ilk değerde her iç yapı kendi süslü parantezini alır, üyeye `.` zinciriyle ulaşılır:

H14_05_ic_ice_tarih.c · satır 50–57

```
50      Ogrenci o = {2301010045, "AYŞE NUR", {14, 9, 2006}, {18, 9, 2023}};
51      Tarih bugun = {.gun = 21, .ay = 12, .yil = 2026};   /* örnek tarih */
52
53      printf("%s doğum: %02d.%02d.%d\n", o.ad,
54             o.dogum.gun, o.dogum.ay, o.dogum.yil);
55      printf("Kayıt anındaki yaşı: %d\n", yas(o.dogum, o.kayit));
56      printf("%02d.%02d.%d itibarıyla yaşı: %d\n",
57             bugun.gun, bugun.ay, bugun.yil, yas(o.dogum, bugun));
```

Şekil 14.6
Örnek 5'in iç içe yapısı: Ogrenci içinde iki Tarih (boyutlar gerçek çalıştırmadan)



Gerçek çıktı: “AYŞE NUR doğum: 14.09.2006”, “Kayıt anındaki yaşı: 17”, “21.12.2026 itibarıyla yaşı: 20”, “sizeof(Tarih) = 12, sizeof(Ogrenci) = 64”. `yas`, doğum gününün o yıl geçip geçmediğine `tarih_karsilastir` ile bakar; yıl **dört hane** tutulur (Bölüm 1).

Örnek 6 ★★ — yapı dizisinde en büyüğün indisi (`H14_06_en_yuksek_ortalama.c`). H10'un deseni değişmedi; değişen, indisin artık bir **kaydın tamamını** seçmesidir:

H14_06_en_yuksek_ortalama.c · satır 27–33

```
27      int enb = 0;
28      for (int i = 1; i < n; i++) {
29          if (ortalama(a[i]) > ortalama(a[enb])) {
30              enb = i;
31          }
32      }
33      return enb;
```

Gerçek çıktı: “En yüksek ortalama: İSMAİL (2301010047) 90.40”, “sizeof sinif = 224, sizeof sinif[0] = 56”. `sinif[k].ad` ve `sinif[k].no` aynı öğrencinindir. Eşit ortalamalarda ilk indis kalır (`>`, `>=` değil).

4.2 Amiral örnek — Örnek 7 ★★: yapı işaretçisi ve -> ile güncelleme

1. Problem. Bir öğrencinin vize ve final notunu güncelleyen bir fonksiyon yazın; ortalama (`ort`) da tutarlı kalsın. Yalnızca okuyan yazdırma fonksiyonu kaydı değiştirememeli ve kopyalamamalı. Değerle aktarılan sürümle farkı gösterin.

Tablo 14.3
Örnek 7'nin girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• kaydın ADRESİ (`Ogrenci *o`) • yeni vize ve final (0–100)	• o->vize ← vize • o->final ← final • o->ort ← 0,4 · vize + 0,6 · final	• main'deki kayıt güncellenmiş • yazdırma: `const Ogrenci *` ile, kopyasız

2–3. Çözümleme ve algoritma. `ort` vize ve finalden **türetilir**; notlardan ayrı değişirse tutarsız kalır. Bu yüzden üç alan tek fonksiyonda birlikte yazılır (Mini-OBS'nin `ders_notu_ata`'sı da öyle). Fonksiyon kayda ancak **adresi** üzerinden ulaşır (H12). **4. C kodu:**

H14_07_isaretci_guncelleme.c · satır 21–55

```c
/* DEĞERLE: o bir kopya; değişiklik çağırana yansımaz */
void guncelle_kopya(Ogrenci o, double vize, double final)
{
    o.vize = vize;
    o.final = final;
    o.ort = 0.4 * vize + 0.6 * final;
    printf("  (kopyada ort = %.2f)\n", o.ort);
}

/* İŞARETÇİYLE: o, kaydın KENDİSİNİ gösterir */
void guncelle(Ogrenci *o, double vize, double final)
{
    o->vize = vize;                 /* (*o).vize = vize; ile aynı */
    o->final = final;
    o->ort = 0.4 * vize + 0.6 * final;
}

/* const işaretçi: okur ama değiştiremez */
void yazdir(const Ogrenci *o)
{
    printf("%lld %-5s vize %5.1f final %5.1f ort %5.2f\n",
           o->no, o->ad, o->vize, o->final, o->ort);
}

int main(void)
{
    Ogrenci a = {2301010045, "AYSE", 70, 85, 79.0};
    yazdir(&a);
    guncelle_kopya(a, 100, 100);
    yazdir(&a);                             /* değişmedi */
    guncelle(&a, 80, 90);
    yazdir(&a);                             /* değişti */

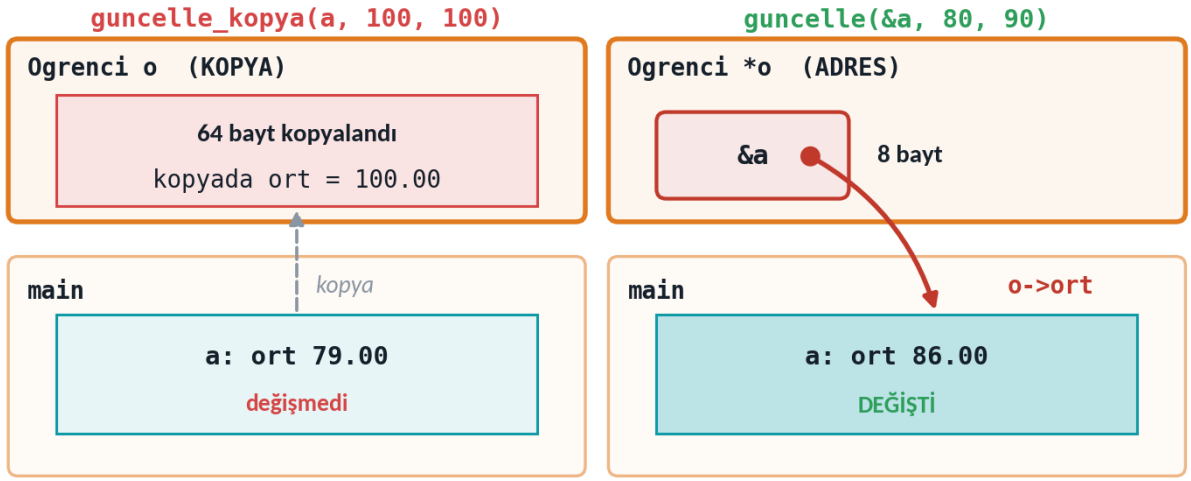
    Ogrenci *p = &a;
    printf("p->ort = %.2f, (*p).ort = %.2f\n", p->ort, (*p).ort);
```

5. Satır satır. Satır 22–27: `o` main'deki a'nın 64 baytlık kopyasıdır; kopya güncellenir, yazdırılır ve dönüşte yok olur. Satır 31–35: parametre bir adrestir; her `o->` main'in kaydına yazar. Satır 39: `const` ile yazdırma fonksiyonu kayda yazamaz; `o->vize = 0;` eklenirse derleyici "assignment of member 'vize' in read-only object" hatası verir. Satır 55: `p->ort` ile `(*p).ort` aynı nesnedir.

Tablo 14.4
Örnek 7'nin 6. adımı, izleme: main'deki a (değişen hücre vurgulu)

Adım	Deyim	a.vize	a.final	a.ort	kopyadaki ort
0	`Ogrenci a = {…}`	70.0	85.0	79.00	–
1	`guncelle_kopya(a, 100, 100)`	70.0	85.0	79.00	**100.00**
2	`guncelle(&a, 80, 90)`	**80.0**	**90.0**	**86.00**	–

Şekil 14.7
Örnek 7: değerle aktarımda kaydın kopyası değişir, işaretçiyle aktarımda main'deki kayıt (gerçek çalıştırmadan)



7. ÖRNEK ÇALIŞTIRMA (GERÇEK ÇIKTI)

```
$ ./isaretci_guncelleme
2301010045 AYSE  vize  70.0 final  85.0 ort 79.00
  (kopyada ort = 100.00)
2301010045 AYSE  vize  70.0 final  85.0 ort 79.00
2301010045 AYSE  vize  80.0 final  90.0 ort 86.00
p->ort = 86.00, (*p).ort = 86.00
Aktarılan: değerle 64 bayt, işaretçiyle 8 bayt
```

8. Test. Değerle aktarımda kopyanın ortalaması 100.00 olurken a'nınki 79.00 kaldı (Şekil 14.7); işaretçiyle aktarımda 80/90 notlarıyla 86.00 oldu (0,4 × 80 + 0,6 × 90); `p->ort` ve `(*p).ort` aynı değeri verdi. Aktarılan veri: değerle 64, işaretçiyle 8 bayt. **9. Değiştir.** (a) `guncelle` geçersiz notta (0–100 dışı) kaydı değiştirmeden `false` döndürsün. (b) Bir yapı dizisinin bütün kayıtlarına %5 artış uygulayan `void hepsini_yukselt(Ogrenci *a, int n)` yazın; `a[i].vize` ile `(a + i)->vize` aynı mıdır?

4.3 Örnek 8–10 (★★–★★★): mini wc, dosya hataları, dolgu deneyi

Örnek 8 ★★ — Mini wc: satır, kelime, karakter, bayt. Kelime sayma H13'ün durum makinesidir (`H13_13_kelime_sayma.c`): bir kelime **dışarıdan içeriye geçişte** sayılır.

H14_08_mini_wc.c · satır 30–35

```
30      int c;                                  /* char DEĞİL: EOF da gelir */
31      while ((c = fgetc(f)) != EOF) {
32          bayt++;
33          if ((c & 0xC0) != 0x80) {           /* UTF-8 devam baytı değil */
34              karakter++;
35          }
```

Kelime durumu `isspace(c)` ile güncellenir: boşlukta `kelime_icinde = false`, boşluk olmayan bir bayt dışarıdayken gelirse kelime sayılır. `c` bir `int`'tir: `fgetc` 0–255 arasındaki baytları ve onlardan ayrı bir değer olan EOF'u döndürür (SEI CERT C FIO34-C). Gerçek çıktı "satır 6, kelime 40, karakter 275, bayt 296" Linux'un `wc`'siyle aynıdır ("6 40 296 veri/ornek_metin.txt"; `wc -m`: 275). Argümansız çağrıda kullanım iletisiyle 2, olmayan dosyada `perror` iletisiyle 1 döner.

Örnek 9 ★★ — Olmayan dosya, izin hatası, klasör. `errno` her kütüphane çağrısında değişebilir; bu yüzden `fopen`'dan hemen sonra saklanır:

H14_09_dosya_hatalari.c · satır 20–26

```
20      errno = 0;
21      FILE *f = fopen(yol, "r");
22      if (f == NULL) {
23          int hata = errno;               /* errno'yu HEMEN sakla */
24          perror(yol);                    /* "yol: ileti" → stderr */
25          fprintf(stderr, "  errno = %d (%s)\n", hata, strerror(hata));
26          return -1;
```

`main` her argüman için `satir_say`'ı çağırır ve okunamayanları sayar; sıradan bir kullanıcıyla (nobody) yapılan gerçek çalıştırmanın iletileri Bölüm 3.6'dadır (hazırlık komutları `ciktilar/` altında). Klasör ilginç bir durumdur: Linux'ta bir klasörü okumak için `fopen` başarılı olur, hata ilk okumada gelir; `fgetc` EOF döndürür ve `ferror` yanıtlar. Yönetici (root) olarak çalıştırıldığında gizli.txt okunabildi: yönetici izin denetimine takılmaz. Hata durumları kullanıcıya **neden** ile birlikte bildirilir; program çökmez ve 1 koduyla biter.

Örnek 10 ★★★ — sizeof(struct) ve dolgu deneyi. `struct Kotu { char harf; double ort; char sube; int no; }` dört üyeyi "küçük, büyük, küçük, orta" sırasıyla, `struct Iyi` aynı üyeleri büyükten küçüğe (`double`, `int`, `char`, `char`) dizer. Program her üyenin `offsetof`'unu ve bir **bayt haritası** yazar:

H14_10_dolgu_deneyi.c · satır 47–53

```
47      isaretle(h, offsetof(struct Kotu, harf), sizeof(char), 'H');
48      isaretle(h, offsetof(struct Kotu, ort), sizeof(double), 'D');
49      isaretle(h, offsetof(struct Kotu, sube), sizeof(char), 'S');
50      isaretle(h, offsetof(struct Kotu, no), sizeof(int), 'N');
51      printf("Kotu: harf %zu, ort %zu, sube %zu, no %zu  |%s|\n",
52             offsetof(struct Kotu, harf), offsetof(struct Kotu, ort),
53             offsetof(struct Kotu, sube), offsetof(struct Kotu, no), h);
```

Gerçek çıktı: “Kotu: harf 0, ort 8, sube 16, no 20 |H.....DDDDDDDDS...NNNN|” (Şekil 14.3 bu çıktıdan çizildi). `ort`'un (double) 8'in katı bir adreste olması için `harf`'ten sonra 7 bayt dolgu girer; `sube`'den sonraki 3 bayt, `no`'nun (int) 4'ün katına düşmesi içindir. `struct Iyi`'de yalnızca sonda 2 bayt dolgu kalır: yapının boyutu en büyük hizalamanın (8) katı olmalıdır ki dizideki bir sonraki elemanın `ort`'u da hizalı olsun. **Değiştir.** Bir `char kod[3]` üyesi ekleyin; önce tahmin edin, sonra ölçün.

4.4 Amiral örnek — Örnek 11 ★★★: CSV'den öğrenci listesinin yüklenmesi

1. Problem. `sinif.csv`'de bir başlık satırı ve her öğrenci için `no;ad;soyad;vize;final` satırları var. Öğrencileri öbekte büyüyen bir yapı dizisine yükleyin; bozuk bir satır programı durdurmasın, satır numarasıyla bildirilip atlansın; sonunda listeyi ve sınıf ortalamasını yazın.

Tablo 14.5
Örnek 11'in girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• dosya yolu (argv[1]; yoksa sinif.csv) • her satır: no;ad;soyad;vize;final	• satırı oku, '\n' ve '\r' sil • başlık ve boş satırı atla • 5 alana böl, her alanı denetle • doluysa realloc ile 2 katına çıkar • kaydı diziye kopyala	• öğrenci listesi ve ortalamalar • atlanan satırlar (stderr) • yüklenen sayı, kapasite

2. Çözümleme. Kısıtlar: alan sayısı tam 5; numara pozitif bir tamsayı; ad ve soyad 31 bayta sığmalı; notlar 0–100 arasında, ondalık ayırıcı nokta. Sınır durumları: başlık ve boş satır, Windows satır sonu (“\r\n”), virgüllü ondalık (85,5), boş alan (“;;”), hiç geçerli satır olmaması (n = 0: ortalama yazılmaz).

3. Algoritma (sözde kod):

Sözde kod

```
her satır için (fgets NULL dönene dek):
    satir_no ← satir_no + 1; sondaki '\r' ve '\n'yi sil
    satir_no = 1 ya da satır boşsa: sonraki satıra geç
    satir_coz(satır, o) başarısızsa: "satır satir_no atlandı" yaz; devam et
    n = kapasite ise: kapasite ← (0 ise 4, değilse 2 × kapasite); realloc
    dizi[n] ← o; n ← n + 1
```

4. C kodu. Alanları bölen ve denetleyen fonksiyon ve `main`'deki okuma döngüsü:

H14_11_csv_yukleme.c · satır 39–64

```
39  static bool satir_coz(char *satir, Ogrenci *o)
40  {
41      char *alan[ALAN_SAYISI];
42      int n = 0;
43      for (char *p = strtok(satir, ";"); p != NULL; p = strtok(NULL, ";")) {
44          if (n == ALAN_SAYISI) {
45              return false;               /* fazla alan */
46          }
47          alan[n++] = p;
48      }
49      char *son;
50      if (n != ALAN_SAYISI) {
51          return false;                   /* eksik alan */
```

```
    }
    o->no = strtoll(alan[0], &son, 10);
    if (son == alan[0] || *son != '\0' || o->no <= 0) {
        return false;
    }
    if (strlen(alan[1]) >= sizeof o->ad
        || strlen(alan[2]) >= sizeof o->soyad) {
        return false;                   /* '\0' için yer kalmalı */
    }
    snprintf(o->ad, sizeof o->ad, "%s", alan[1]);
    snprintf(o->soyad, sizeof o->soyad, "%s", alan[2]);
    return not_coz(alan[3], &o->vize) && not_coz(alan[4], &o->final);
}
```

H14_11_csv_yukleme.c · satır 77–88

```
    while (fgets(satir, sizeof satir, f) != NULL) {
        satir_no++;
        satir[strcspn(satir, "\r\n")] = '\0';   /* '\n' ve '\r' sil */
        if (satir_no == 1 || satir[0] == '\0') {
            continue;                           /* başlık, boş satır */
        }
        Ogrenci o = {0};
        if (!satir_coz(satir, &o)) {
            fprintf(stderr, "Uyarı: %s satır %d biçim hatası, atlandı\n",
                    yol, satir_no);
            atlanan++;
            continue;
```

5. Satır satır. Satır 43: strtok her çağrıda bir sonraki alanın başını döndürür ve ayırıcının yerine '\0' yazar; fazla alan dizinin dışına yazmadan reddedilir (satır 44). Satır 53–54: “metnin tamamı sayı mı?” denetimi (H13). Satır 57–62: ad, '\0' için yer kalacak biçimde sınırlı kopyalanır. Satır 79: `strcspn` ilk '\r' ya da '\n'nin indisini verir; oraya '\0' yazmak ikisini birden siler. Döngünün kalanında (satır 90–100) dizi doluysa kapasite ikiye katlanır ve realloc sonucu önce geçici işaretçiye alınır (H12_12); `dizi[n++] = o;` yapı atamasıyla kaydın bütün baytlarını kopyalar.

Tablo 14.6

Örnek 11'in 6. adımı, izleme (veri/sinif.csv; değişen hücre vurgulu)

Satır	İçerik (kısaltılmış)	Sonuç	n	kapasite
1	no;ad;soyad;vize;final	başlık: atla	0	0
2	2301010045;AYŞE NUR;…;70;85	yüklendi (realloc: 0 → 4)	1	**4**
3-4	MEHMET ALİ, İSMAİL	yüklendi	3	4
5	2301010048;CAN;YILMAZ;85,5;90	**atlandı: “85,5” sayı değil**	3	4
6	2301010049;ZEYNEP;ÇELİK;70;80	yüklendi	4	4
7	2301010050;;ÇINAR;95;98	**atlandı: strtok 4 alan buldu**	4	4
8	2301010051;ŞULE;DOĞAN;52;58	yüklendi (realloc: 4 → 8)	5	**8**
9-10	ILGAZ, ELİF	yüklendi	7	8

7. ÖRNEK ÇALIŞTIRMA (GERÇEK; UYARILAR STDERR'E YAZILDI)

```
$ ./csv_yukleme veri/sinif.csv
Uyarı: veri/sinif.csv satır 5 biçim hatası, atlandı
Uyarı: veri/sinif.csv satır 7 biçim hatası, atlandı
2301010045   79.00  AYŞE NUR YILMAZ
2301010046   46.80  MEHMET ALİ KAYA
2301010047   90.40  İSMAİL ÖZTÜRK
2301010049   76.00  ZEYNEP ÇELİK
2301010051   55.60  ŞULE DOĞAN
2301010052   36.00  ILGAZ ŞAHİN
2301010053   97.00  ELİF KARA
7 öğrenci yüklendi (kapasite 8), 2 satır atlandı.
Sınıf ortalaması: 68.69
```

8. Test. Gerçek çalıştırmada beş durum sınandı: normal satır (AYŞE NUR, ortalama 79.00), başlık ve boş satır (uyarısız atlandı), virgüllü ondalık (satır 5 uyarıyla atlandı), boş ad alanı (satır 7 atlandı) ve büyüme (7 geçerli satır: kapasite 4 → 8). **9. Değiştir–genişlet.** (a) Bozuk satırda **durup** hiçbir kaydı yüklemeyen (hepsi-ya-hiç) sürüm yazın: Mini-OBS böyle davranır, neden? (b) Mükerrer numarayı reddedin. (c) strtok yerine Mini-OBS'nin `alanlara_bol`'unu kullanıp boş adı "eksik alan"dan ayrı bir iletiyle bildirin.

4.5 Örnek 12 (★★★): ikili dosyaya yapı dizisi yazma ve okuma

Program üç `Kayit`'i (`int no; char ad[8]; double ort;`, 24 bayt) `memset` ile sıfırlanmış olarak `"wb"` kipinde tek `fwrite` ile yazar (`fwrite(yaz, sizeof yaz[0], 3, f)`), aynı veriyi karşılaştırma için metin olarak da yazar, sonra `"rb"` ile okur:

H14_12_ikili_dosya.c · satır 68–80

```
68      Kayit oku[10];
69      size_t n = fread(oku, sizeof oku[0], 10, f);    /* 10 istendi */
70      printf("fread %zu kayıt döndürdü\n", n);
71      for (size_t i = 0; i < n; i++) {
72          printf("  %d %-6s %.2f\n", oku[i].no, oku[i].ad, oku[i].ort);
73      }
74      long konum = 2 * (long)sizeof(Kayit);           /* 3. kaydın başı */
75      Kayit k;
76      if (fseek(f, konum, SEEK_SET) == 0 && fread(&k, sizeof k, 1, f) == 1) {
77          printf("3. kayıt (fseek %ld): %d %s %.2f\n", konum, k.no, k.ad,
78                 k.ort);
79      }
80      fclose(f);
```

GERÇEK ÇIKTI (ONALTILI DÖKÜM BÖLÜM 3.10'DA)

```
$ ./ikili_dosya
sizeof(Kayit) = 24, dosya = 72 bayt
fread 3 kayıt döndürdü
  1 EMRE   79.00
  2 CAN    88.35
  3 ZEYNEP 52.00
3. kayıt (fseek 48): 3 ZEYNEP 52.00
```

`fread`'den 10 kayıt istendi, dosyada 3 vardı: dönüş değeri **okunabilen** sayıdır ve döngü bu sayıyla kurulur. 3. kayda `fseek` ile doğrudan gidildi; metin dosyasında bunun için baştan satır saymak gerekirdi. `fseek(f, 0, SEEK_END)` + `ftell` ile dosya boyutunu bulmak POSIX ve Windows'ta çalışır, ama standart ikili akışlarda `SEEK_END` desteğini zorunlu tutmaz (7.21.9.2). Dosyanın onaltılı dökümü ve dolgu baytları Bölüm 3.10'dadır.

5. Hata Avcısı

Kodlar `kodlar/hata_avcisi/` altındadır; her hatalı dosyanın başında "BU DOSYA KASITLI HATA İÇERİR" yazar ve yanında `_duzeltilmis.c` sürümü vardır. İletiler ve davranışlar bu sistemde (GCC 13, glibc) **gerçekten** gözlenenlerdir (Tablo 14.7).

Tablo 14.7
Hata Avcısı: hata, ileti/davranış, neden, çözüm

#	Hata	İleti / gözlenen davranış	Neden	Çözüm
HA1	`if (a == b)` (iki yapı)	derleme hatası: "invalid operands to binary == (have 'Nokta' and 'Nokta')"	== yapılar için tanımlı değil	alan alan karşılaştıran fonksiyon
HA2	`p.ort` (p işaretçi), `o->ort` (o yapı), `*p.ort`	"'p' is a pointer; did you mean to use '->'?"; "invalid type argument of '->' (have 'Ogrenci')"	. yapıyla, -> işaretçiyle; . önce bağlanır	`p->ort`, `o.ort`, `(*p).ort`
HA3	`fopen` dönüşü denetlenmedi; klasör yok	Segmentation fault, çıkış durumu 139 (tanımsız davranış)	NULL akışa fprintf	`if (f == NULL) { perror(yol); return 1; }`
HA4	`fclose` yok, program `abort()` ile bitti	"0 gunluk.txt": veri tamponda kaldı	anormal sonlanmada tampon boşaltılmaz (uygulamaya bağlı)	işi biten akışı hemen kapat; düzeltilmişte "72 gunluk.txt"
HA5	`while (!feof(f))`	"4 kayıt, toplam 335"	feof okuma çarptıktan SONRA doğru olur	`while (fgets(…) != NULL)`
HA6	eklemek için `fopen(…, "w")`	iki eski kayıt silindi; dosyada yalnızca "2301010048 CAN YILMAZ"	w dosya varsa uzunluğu sıfırlar	`"a"` (ya da yanlışlıkla üzerine yazmamak için `"wx"`)

HA2'nin üç hatası (`p.ort`, `o->ort`, `*p.ort`) tek bir dosyadadır; GCC `p.ort`'ta ve `*p.ort`'ta düzeltmeyi kendisi önerir ("'p' is a pointer; did you mean to use '->'?"), `o->ort` için "invalid type argument of '->' (have 'Ogrenci')" der. `*p.ort` öncelik tuzağıdır: `.` (son ek operatörü) tekli `*`'dan önce bağlanır, yani ifade `*(p.ort)` olur.

HA4: kaybolan üç kayıt. Program "Üç kayıt yazıldı (sanıyoruz)." yazdı ve `abort()` ile sonlandı (çıkış durumu 134); `gunluk.txt` oluşturulmuştu ama boştu: kayıtlar C kütüphanesinin tamponundaydı ve işletim sistemine hiç gitmedi (3.7). Düzeltilmiş sürümde aynı `abort()` var, ama `fclose` ondan önce çağrıldığı için 72 bayt diske ulaştı. **HA6: "w" kipinin sessiz silmesi.** Derleyici de çalışma zamanı da hiçbir şey söylemez; iki kayıtlık `ogrenciler.txt` HA6'dan sonra tek satıra indi, düzeltilmiş sürüm (`"a"`) üç satır bıraktı.

6. Mühendis Gözüyle: Veri Modelleme, Dayanıklılık ve Güvenlik

- **Veri modelleme uzun yaşar.** Y2K'nın dersi: bir alanın boyutu ve biçimi onlarca yıl değişmeyebilir. Yılı dört haneyle saklayın; türetilebilen bir değeri (Mini-OBS'de `ort`) yalnızca tek bir fonksiyonda hesaplayın, harf notu gibi kurallara bağlı değerleri hiç saklamayın: kural değişirse saklanan değer yanlış kalır.

- **Dosyadan gelen veri güvenilmez girdidir.** Elle düzenlenmiş, yarıda kesilmiş ya da başka bir programın yazdığı bir dosya her alanında yanlış değer taşıyabilir. Mini-OBS her alanı klavye girdisiyle aynı kurallarla denetler, bozuk dosyayı **yarım yüklemez** ve program hiç başlamaz (çıkış kodu 1).
- **Dayanıklı kaydetme.** Doğrudan veri dosyasının üzerine yazmak, yazma yarıda kalırsa eski veriyi de yok eder. Mini-OBS'nin `sinif_kaydet`'i önce `ogrenciler.csv.tmp`'ye yazar, `fclose`'un sonucunu denetler, ancak her şey yolundaysa `rename` ile geçici dosyayı eski dosyanın yerine koyar. Yazma hatasında geçici dosya silinir, eski veri olduğu gibi kalır; yeni dosya yerine konamazsa geçici dosya **silinmez** ve kullanıcıya yeni verinin orada olduğu söylenir (7.1). İlke: hiçbir hata yolu iki kopyayı birden silmemeli.
- **Taşınabilirlik.** CSV her makinede okunur; ikili dosya yalnızca aynı derleyici ve aynı mimaride güvenle okunur. Mini-OBS ikili başlığa `sizeof(Ogrenci)` yazar ve farklı bir boyutla karşılaşırsa dosyayı reddeder.
- **Güvenli kodlama** (SEI CERT C; Carnegie Mellon University Software Engineering Institute, 2016; Seacord, 2013). ERR33-C: kütüphane hatalarını algılayıp ele alın (her `fopen`, `fclose`, `fwrite`, `fread` dönüşü; HA3, Örnek 9). FIO34-C: okunan karakteri EOF'tan ayırın (`int c = fgetc(f)`, Örnek 8). FIO39-C: aynı akışta okuma ile yazma arasında `fflush` ya da konumlama yapın (Alıştırma 14.10). FIO42-C: gerekmeyen dosyaları kapatın (HA4). EXP42-C: dolgu verisini karşılaştırmayın (H14_K04). DCL39-C: yapı güven sınırını geçerken bilgi sızdırmayın (kayıtları sıfırlayarak yazmak; Örnek 12, Mini-OBS).

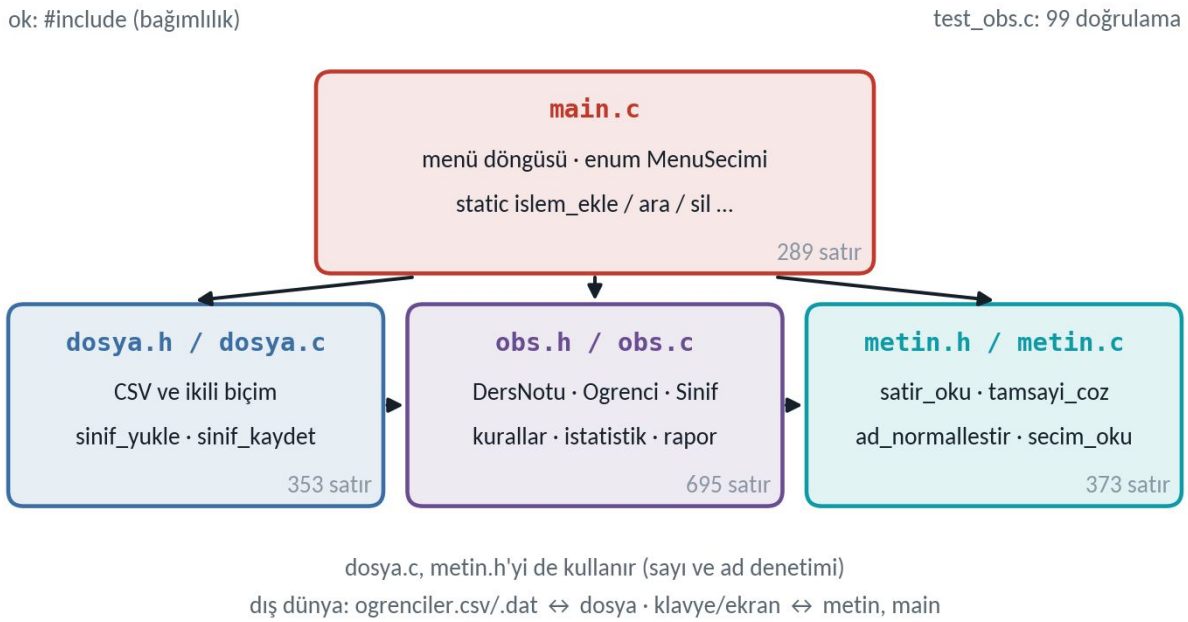
Test. Mini-OBS'nin testleri disk adı gerektirmeden `tmpfile()` ile açılan geçici akışlar kullanır: `csv_yaz` ve `ikili_yaz` bir `FILE *` aldığı için aynı fonksiyon hem gerçek dosyada hem testte çalışır. Okuma–yazma **gidiş-dönüş** testi (yaz, geri oku, alan alan karşılaştır) dosya kodunun en güçlü testidir; bozuk girdiler (eksik alan, virgüllü ondalık, kesik ikili dosya) ayrı ayrı sınanır.

7. Mini-OBS v1.0: Bütünleştirme (Örnek 13)

Merdivenin 13. basamağı dönemin sentezidir. v0.13'te bir öğrencinin verisi altı paralel blokta duruyordu ve program kapanınca kayboluyordu. v1.0'da veri **tek bir kayıt tipinde** toplanır, **dosyada kalıcı** olarak saklanır ve kod **dört modüle** ayrılır; hesap kuralları (RTEÜ harf tablosu, %40 vize + %60 final, final barajı 50, şartlı DC, iki haneye yuvarlanan YANO) değişmedi (Recep Tayyip Erdoğan Üniversitesi, 2025). Kaynaklar `kodlar/proje/` altında; ayrıntılı belirtim (`mini_obs_v1_0.md`) ve bütün gerçek çıktılar proje klasöründedir.

Modüller ve v0.13'ten farklar (Şekil 14.8)**.** `metin` girdi ve metin işlemeyi toplar ve öğrenci kaydını hiç bilmez; `obs` kayıt tiplerini ve iş kurallarını, `dosya` kalıcılığı içerir; `main` yalnızca menü akışıdır (38 satır: yükleme `veriyi_yukle`, seçim `islem_yap`'taki `switch`, işlemler `static islem_*` fonksiyonları). Bir modül yalnızca ihtiyaç duyduğu başlıkları ekler: yüksek uyum, düşük bağlaşım (H9). Altı ayrı `realloc` yerine tek `realloc`; 7 yerine 10 menü seçeneği (`enum MenuSecimi`; güncelle, sil, kaydet yeni); 2B dizi parametreleri yerine `const Sinif *`; testler 99 doğrulama (37 kural + 22 metin + 24 yapı + 16 dosya). Karşılaştırma tablosu `mini_obs_v1_0.md`'nin 1. bölümündedir.

Şekil 14.8
Mini-OBS v1.0 modül mimarisi: oklar #include bağımlılıklarıdır; satır sayıları gerçek kaynaklardan



proje/obs.h · satır 31–36

```
31  typedef struct {
32      long long no;
33      char ad[AD_BOYUT];      /* normalleştirilmiş, '\0' ile biten UTF-8 */
34      char soyad[SOYAD_BOYUT];
35      DersNotu ders[DERS_SAYISI];
36  } Ogrenci;
```

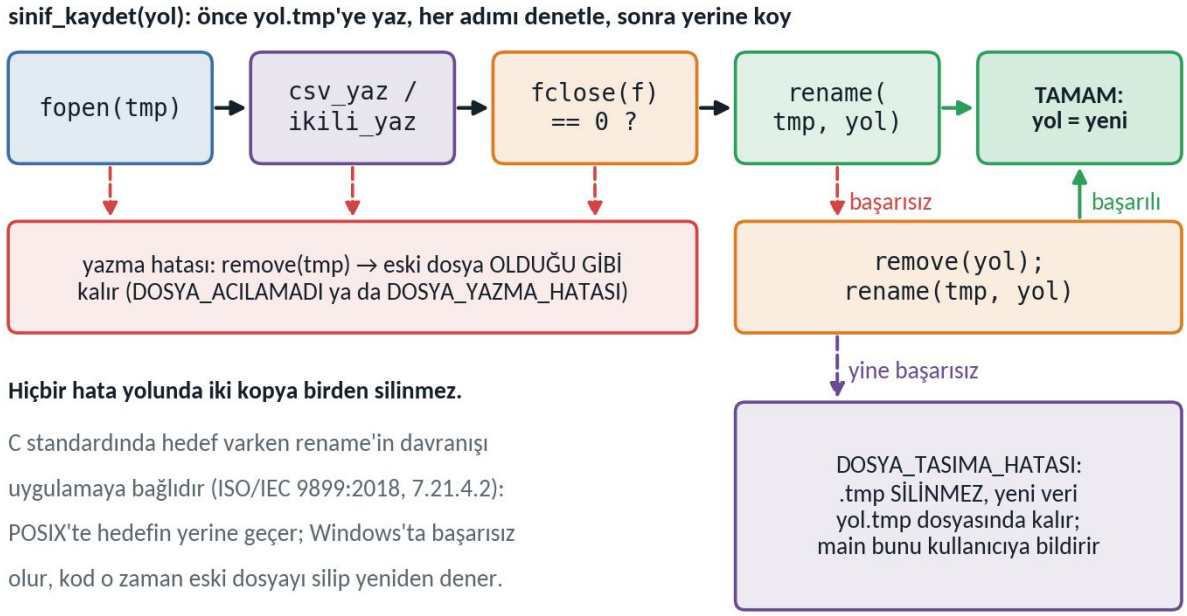
Tasarım kararları. (1) Her ders bir `DersNotu { vize, final, ort }`'tur: `ort` türetilmiş bir değerdir ve yalnızca `ders_notu_ata` hesaplar; harf notu kurala bağlı olduğu için **hiç saklanmaz**. (2) Kayıtlar öbekte tek blokta yan yana durur: `Sinif { Ogrenci *kayit; int n, kapasite; }` (`sizeof(Ogrenci)` = 144, dolgusuz; `Sinif` 16 bayt); `sinif_ekle` doluysa kapasiteyi ikiye katlar ve `s->kayit[s->n] = *o;` ile kaydın bütün üyelerini kopyalar; `sinif_sil` H10'un kaydırma desenini tek yapı atamasıyla uygular. (3) İşlem fonksiyonları girdi biterse `false` döndürür; bütün çıkışlar tek temizlik yolundan (`sinif_birak`) geçer.

7.1 Dosya biçimleri, testler ve kabul

Biçimi dosya uzantısı seçer: `./mini_obs ogrenciler.csv` ya da `./mini_obs ogrenciler.dat`. **CSV:** başlık satırı `no;ad;soyad;ALG_vize;ALG_final;MAT_vize;MAT_final;FIZ_vize;FIZ_final`; notlar `%.15g` ile yazılır: en çok 15 anlamlı basamaklı bir değer (85.5, 90.25 gibi klavyeden girilen notlar) okuma–yazma döngüsünde birebir korunur. `csv_oku` dosya başındaki UTF-8 imzasını (BOM: `EF BB BF`; Excel'in "CSV UTF-8" biçimi) atlar, her satırı `alanlara_bol` ile böler, alanları denetler, **mükerrer numarayı** reddeder ve bozuk satırda okumayı satır numarasıyla durdurur. **İkili:** 16 baytlık başlık (imza `'M' 'O' 'B' 'S'`, sürüm 1, kayıt boyutu `sizeof(Ogrenci)` = 144 ve kayıt sayısı; üçü de küçük-uçlu `int32_t`) ve ardından kayıtlar, bellekteki baytlarıyla. T6'nın dosyasının ilk 16 baytı: `4d 4f 42 53 01 00 00 00 90 00 00 00 01 00 00 00`.

`ikili_oku` başlıktaki kayıt sayısına güvenip tek seferde dev bir blok ayırmaz: kayıtları **tek tek** okur ve doğrular (ad dizisinin içinde '\0' var mı, notlar 0–100 arasında mı); `ort`'u dosyadan almaz, yeniden hesaplar; kayıtlardan sonra fazladan bayt varsa dosyayı reddeder. `sinif_yukle` kayıtları önce **geçici bir sınıfa** okur; hepsi geçerliyse yapı atamasıyla (`*s = gecici;`) mevcut verinin yerine koyar, değilse mevcut veri değişmez. Dosya yoksa (`errno == ENOENT`) bu bir hata değil, ilk çalıştırmadır. Kaydetme Şekil 14.9'teki sırayla yapılır. C standardında hedef varken `rename`'in davranışı uygulamaya bağlıdır (ISO/IEC, 2018, 7.21.4.2); Windows'ta başarısız olduğu için (Microsoft, 2022) kod eski dosyayı silip yeniden dener. İkinci deneme de başarısızsa `DOSYA_TASIMA_HATASI` döner, `.tmp` silinmez ve `main` "yeni veri <yol>.tmp dosyasında" der.

Şekil 14.9
Mini-OBS v1.0'ın güvenli kaydetme akışı (sinif_kaydet): yazma hatasında eski dosya, taşıma hatasında yeni veri korunur



Test planı. Yedi oturum (T1–T6b) ile kesik/imzası bozuk ikili dosyalar, kullanım hataları, BOM'lu ve ANSI kodlu CSV ve bir taşıma hatası gerçekten çalıştırıldı; senaryo tablosu `mini_obs_v1_0.md`'de (bölüm 7), tam çıktılar `mini_obs_v1_0_ciktilar.txt`'dedir. T1'in CSV dosyasının ilk kaydı `2301010045;AYŞE NUR;YILMAZ;70;85;90;95;40;60`; T6'nın ikili dosyası 160 bayttır. **v0.13 ile karşılaştırma:** aynı sekiz öğrencinin notları v0.13'e klavyeden, v1.0'a CSV dosyasından verildi; "Ders istatistikleri" ve "Harf notu dağılımı" bölümleri `diff` ile **birebir aynı** çıktı. **Bellek:** `make asan` rapor üretmedi; Valgrind her oturumda "All heap blocks were freed -- no leaks are possible" dedi (ör. T2: 9 allocs, 9 frees, 14,680 bytes allocated).

> MİNİ-OBS PROJESİ · **Kabul ölçütleri (özet; ayrıntısı mini_obs_v1_0.md, bölüm 9–10)**
>
> - Sıfır uyarı; `make`, `make test`, `make ornek`, `make asan`, `make valgrind` çalışır; 99 doğrulamanın hepsi geçer; menünün on işlevi çalışır.
> - İkili dosyada gidiş-dönüş kayıpsız, CSV'de 15 anlamlı basamağa kadar kayıpsızdır; bozuk dosyada program başlamaz (1), hatalı kullanımda 2 ile çıkar; algılanan hiçbir hata veri dosyasını yarım yazılmış bırakmaz ve iki kopyayı birden silmez.

8. Alıştırmalar, Atölye ve Final Projesi

8.1 Alıştırmalar

Kodlar `kodlar/alistirmalar/` altındadır (kesitlerin satır numaraları dosyalarla aynıdır). Çözümler öğretim üyesindedir.

Alıştırma 14.1 ★☆☆ *Çıktı tahmini*

Programı çalıştırmadan çıktısını tahmin edin. Her satırdan sonra a ve b'nin üyelerini bir tabloya yazın (`Oyuncu` yapısı `char ad[8]; int puan;` üyelerinden oluşur; `bonus(Oyuncu o)` ve `bonus_p(Oyuncu *o)` puanı 10 artırır).

alistirmalar/H14_A01_kopya_tahmin.c · satır 24–35

```
24  int main(void)
25  {
26      Oyuncu a = {"ADA", 50};
27      Oyuncu b = a;
28      bonus(a);
29      bonus_p(&b);
30      b.ad[0] = 'O';
31      Oyuncu *p = &a;
32      p->puan *= 2;
33      printf("%s %d / %s %d\n", a.ad, a.puan, b.ad, b.puan);
34      return 0;
35  }
```

Alıştırma 14.2 ★★☆ *Çıktı tahmini (dolgu)*

`alistirmalar/H14_A02_dolgu_tahmin.c` üç yapı tanımlar: `struct A { char a; double b; char c; };`, `struct B { double b; char a; char c; };` ve `struct C { char a; char c; int i; };`. x86-64 Linux GCC için `sizeof(struct A)`, `sizeof(struct B)`, `sizeof(struct C)` ve `offsetof(struct A, c)` değerlerini tahmin edin (program dördünü tek satırda yazar) ve her yapının bayt haritasını çizin.

Alıştırma 14.3 ★★☆ *Hata bul*

Program üç not ekleyip ortalamayı yazmalıydı; oysa "Not sayısı: 0" yazıyor ve derleyici uyarı vermiyor. Hatayı yığın çerçeveleriyle açıklayıp düzeltin.

alistirmalar/H14_A03_hata_bul_not_ekle.c · satır 18–31

```
18  void not_ekle(Karne k, double puan)
19  {
20      if (k.n < AZAMI) {
21          k.notlar[k.n] = puan;
22          k.n++;
23      }
24  }
25
26  int main(void)
27  {
28      Karne k = {{0}, 0};
29      not_ekle(k, 70);
30      not_ekle(k, 85);
31      not_ekle(k, 90);
```

Alıştırma 14.4 ★★☆ *Hata bul (üç hata)*

Program kayitlar.txt'ye bir satır EKLEYİP satırları saymalıydı. İki satırlık bir dosyayla çalıştırıldığında "Dosyada 2 satır var." yazdı ve dosyada tek satır kaldı. Üç hatayı (biri yalnızca belirli koşullarda ortaya çıkar) bulun ve düzeltin.

`alistirmalar/H14_A04_hata_bul_dosya.c · satır 11–27`

```c
int main(void)
{
    FILE *f = fopen("kayitlar.txt", "w");
    fprintf(f, "2301010048 CAN\n");
    fclose(f);

    f = fopen("kayitlar.txt", "r");
    char satir[64];
    int n = 0;
    while (!feof(f)) {
        fgets(satir, sizeof satir, f);
        n++;
    }
    fclose(f);
    printf("Dosyada %d satır var.\n", n);
    return 0;
}
```

Alıştırma 14.5 ★★☆ *Parsons*

Karışık satırları sıraya dizip girintileyin: `numarala(yol)` dosyanın satırlarını "1: …" biçiminde yazsın, satır sayısını döndürsün; açılamazsa nedenini yazıp −1 döndürsün. Karışık 15 satır `alistirmalar/H14_A05_parsons_numarala.c` dosyasındadır (satır 8–22; bu hâliyle derlenmez).

Alıştırma 14.6 ★★☆ *Kod tamamla*

`alistirmalar/H14_A06_kod_tamamla_ikili.c` dosyasındaki beş boşluğu (`____`) doldurun: iki `fopen` kipi, `fwrite`'ın eleman boyutu, `fread`'in hedefi ve yazdırılan üye. Program "2 kayıt: (1, 2) (3, 4)" yazmalıdır (bu hâliyle derlenmez).

Alıştırma 14.7 ★★☆ *Kod yaz*

`Tarih` yapısı için `bool tarih_gecerli_mi(Tarih t)` ve bir sonraki günü **yapı olarak döndüren** `Tarih sonraki_gun(Tarih t)` yazın (artık yıl: 4'e bölünen, ama 100'e bölünüp 400'e bölünmeyenler hariç). 28.02.2024, 31.12.2026, 31.04.2026 ve 29.02.1900 ile sınayın.

Alıştırma 14.8 ★★☆ *Kod yaz*

`Kitap { kod, ad, yil }` yapısını bir CSV dosyasının **sonuna** ekleyen `bool kitap_ekle(const char *yol, const Kitap *k)` ve dosyayı listeleyen `int kitaplari_listele(const char *yol)` yazın. Satırları `sscanf(satir, "%11[^;];%47[^;];%d", …)` ile ayrıştırın; genişlikler neden tampon boyutundan bir eksik?

Alıştırma 14.9 ★★☆ *Kod yaz*

"no vize final" satırlarından oluşan bir dosyayı `fscanf` ile okuyup kayıt sayısını, en yüksek ve en düşük ortalamalı öğrenciyi ve sınıf ortalamasını yazın; bozuk bir kayıtta okumayı durdurup kaçıncı kayıt olduğunu bildirin.

Alıştırma 14.10 ★★★ *Kod yaz*

İkili bir kayıt dosyasında k. kaydın ortalamasını **yerinde** güncelleyen `bool kayit_guncelle(const char *yol, long k, double yeni_ort)` yazın: `"r+b"` ile açın, `fseek` ile kaydın başına gidin, okuyun, değiştirin, yeniden aynı konuma gidip yazın. Okuma ile yazma arasındaki `fseek` neden zorunlu? Olmayan bir kayıt (k ≥ kayıt sayısı) nasıl fark edilir?

Alıştırma 14.11 ★★★ *Kod yaz (meydan okuma)*

Bir yapı dizisini CSV'ye yazıp geri okuyan ve alan alan karşılaştıran bir gidiş-dönüş testi yazın. Önce `%.15g`, sonra `%.17g` ile deneyin; ortalaması `0.1 + 0.2` olan bir kayıtla iki biçimin sonucunu karşılaştırıp açıklayın.

8.2 Atölye 1 — Final projesi atölyesi: kod incelemesi ve eşli hata ayıklama

Kod incelemesi (code review, 15 dk). Eşler projelerinin (ya da Mini-OBS'nin) birer modülünü değiş tokuş eder ve aşağıdaki listeyle inceler; her bulgu satır numarasıyla yazılır, kodun yazarı bulguyu yanıtlar. Amaç kişiyi değil kodu değerlendirmektir.

- ☐ Derleme: `gcc -std=c17 -Wall -Wextra -pedantic` ile sıfır uyarı; `make` temiz bir klasörde çalışıyor.
- ☐ Her `fopen`, `malloc`/`realloc`, `fclose`, `fread`/`fwrite`, `scanf`/`fgets` dönüşü denetleniyor; hata iletisi nedeni söylüyor.
- ☐ Okuma döngüleri okuma fonksiyonunun dönüşüne bağlı (`while (!feof(f))` yok); her açılan akış kapatılıyor.
- ☐ Yapılar değiştirilmeyecekse `const T *` ile aktarılıyor; türetilmiş alanlar tek fonksiyonda hesaplanıyor.
- ☐ Adlar anlamlı, sabitler adlandırılmış (enum, `#define`, `static const`); fonksiyonlar tek iş yapıyor ve 40 satırı aşmıyor.
- ☐ Dosyadan ve klavyeden gelen her alan aralık ve biçim yönünden doğrulanıyor; bozuk dosya veri kaybına yol açmıyor.

Eşli hata ayıklama (15 dk). Bir öğrenci "sürücü" (klavyede), diğeri "gözcü"dür; 7–8 dakikada bir rol değişir. Öğretim üyesinin verdiği hatalı bir sürümde (ör. `while (!feof(f))`, unutulmuş `fclose`, değerle aktarılan güncelleme fonksiyonu) önce hatayı **yeniden üretin** (girdi dosyasıyla), sonra tahmin–gözlem döngüsüyle daraltın: `printf` yerine `gdb` ile kesme noktası ve `print *o`, bellek hataları için `-fsanitize=address,undefined` ve `valgrind --leak-check=full`. Düzeltmeden sonra hatayı yakalayan bir test ekleyin.

8.3 Atölye 2 — Proje sunumları: canlı demo, mimari ve test senaryoları

Her ekip 8 dakika sunar, 4 dakika soru yanıtlar. Önerilen akış: (1) **Problem ve kapsam** (1 dk): ne yapıyor, kim kullanır? (2) **Canlı demo** (3 dk): temiz bir klasörde `make`; örnek veri dosyasıyla açılış, en az bir

ekleme, bir arama, bir hata durumu (geçersiz girdi ya da bozuk dosya) ve kaydet–çık–yeniden aç döngüsü; veri kalıcı mı? (3) **Mimari** (2 dk): modül diyagramı (Şekil 14.8 gibi), veri modeli ve bir tasarım kararının gerekçesi. (4) **Testler** (2 dk): test tablosu, `make test`, `make asan`/`valgrind` çıktısı. Demo canlı yapılır; çalışmazsa kayıtlı terminal çıktısı yedek olarak gösterilir.

8.4 Atölye 3 — Akran değerlendirmesi ve rubrik temelli geri bildirim

Her öğrenci iki başka ekibin sunumunu ve kodunu rubriğin beş ölçütüyle değerlendirir (Tablo 14.8). Her ölçüt için 1–4 arası bir düzey işaretlenir; puan yerine **gözleme dayalı** cümle yazılır: "T5'te bozuk dosyada program başlamadı ve satır numarasını verdi" gibi. Form iki güçlü yön ve bir somut öneriyle biter. Akran puanları ekibe geri bildirim içindir; not, öğretim üyesinin rubrik değerlendirmesidir.

Tablo 14.8
Final projesi rubriğinin ölçütleri ve ağırlıkları (ayrıntılı düzey tanımları final projesi yönergesinde)

Ölçüt	Ağırlık	Akranın sorduğu soru
İşlevsel doğruluk	%30	Bütün menü işlevleri beklenen çıktıyı üretiyor mu; sınır durumları doğru mu?
Algoritma ve tasarım	%20	Yapı dizisi, dinamik bellek ve modül ayrımı anlamlı mı?
Kod kalitesi	%20	Adlar, biçim, yorumlar, uyarısız derleme, kod standardı
Sağlamlık ve güvenlik	%15	Girdi ve dosya doğrulama, hata denetimi, sızıntı yokluğu
Test ve sunum	%15	Test senaryoları, canlı demo, tasarım kararlarının gerekçesi

Final projesi. Varsayılan proje Mini-OBS v1.0'ın **kendi gerçeklemenizdir**; banka/ATM, kütüphane ödünç takibi, stok ve satış, otopark yönetimi, skor tablosu dosyada tutulan metin tabanlı bir oyun ya da şifreleme aracı da seçilebilir. Her seçenek ortak asgari gereksinimleri karşılar: **yapı dizisi, dinamik bellek, dosya kalıcılığı, çok dosyalı yapı, menü ve girdi doğrulama**. Teslim paketi kaynaklar, Makefile, README, test girdileri ve beklenen çıktılar, örnek veri dosyası ve kısa bir tasarım belgesidir; işlevsel gereksinim tabloları, rubriğin düzey tanımları, akran formu, akademik dürüstlük ve yapay zekâ beyanı final projesi yönergesindedir. **Teslim ve sunum tarihleri ders sayfasında duyurulur.**

9. Kendini Değerlendirme

- ☐ İlişkili verileri bir `struct`'ta toplayabiliyor, typedef, iç içe yapı ve yapı dizisi kullanabiliyorum; yapı atamasının dizi üyeleri de kopyaladığını ve yapıların neden `==` ile karşılaştırılamadığını açıklayabiliyorum.
- ☐ Yapıları fonksiyonlara değerle ve `const` işaretçiyle aktarabiliyor; `.` ile `->` arasındaki farkı ve `*p.x` tuzağını açıklayabiliyorum.
- ☐ Bir yapının `sizeof`'unu dolguyla birlikte tahmin edebiliyor, üye sırasının etkisini gösterebiliyorum.
- ☐ Doğru kipte dosya açıp `NULL`, `fclose` ve okuma dönüşlerini denetleyebiliyor; `while (!feof(f))` hatasını açıklayabiliyorum.
- ☐ CSV ve ikili dosyaya yapı dizisi yazıp geri okuyabiliyor, onaltılı dökümü yorumlayabiliyor ve ikili dosyanın neden taşınabilir olmadığını söyleyebiliyorum.
- ☐ Modüler, menü güdümlü, verisini dosyada saklayan bir programı test edip sunabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · Bu haftanın yapı taşları AP-2'de nerede kullanılacak?

- Yapı dizisini bir alana göre (ortalama, soyad) sıralamak AP-2'deki **sıralama algoritmalarının** doğal uygulamasıdır; eşit anahtarlı kayıtların ilk sıralarını koruyup korumadığı **kararlılık** (stability) kavramını doğurur.
- Kendi tipinden bir işaretçi taşıyan yapı, bağlı liste ve diğer dinamik veri yapılarının kapısıdır (aşağıda).
- Dosyadan büyük veri okumak, AP-2'de algoritmaların gerçek veri setlerinde **performansını ölçmeyi** sağlar (H7'deki `clock()` ölçümüyle birlikte).

H14_K15_dugum.c · satır 14–22

```c
struct Dugum {
    int veri;
    struct Dugum *sonraki;          /* bir sonraki düğümün ADRESİ */
};

int main(void)
{
    struct Dugum b = {20, NULL};    /* NULL: zincirin sonu */
    struct Dugum a = {10, &b};
```

Bir yapı kendi tipinden bir **üye** taşıyamaz (boyutu sonsuz olurdu), ama kendi tipinden bir **işaretçi** taşıyabilir. `H14_K15_dugum.c` iki düğümü elle bağlar ve `->` zincirini izler: "a.sonraki->veri = 20", "a.sonraki->sonraki == NULL". Liste işlemleri (ekleme, silme, dolaşma) AP-2'nin konusudur.

MERAK KUTUSU · 2038: işaretli 32 bitlik saniye sayacının sonu

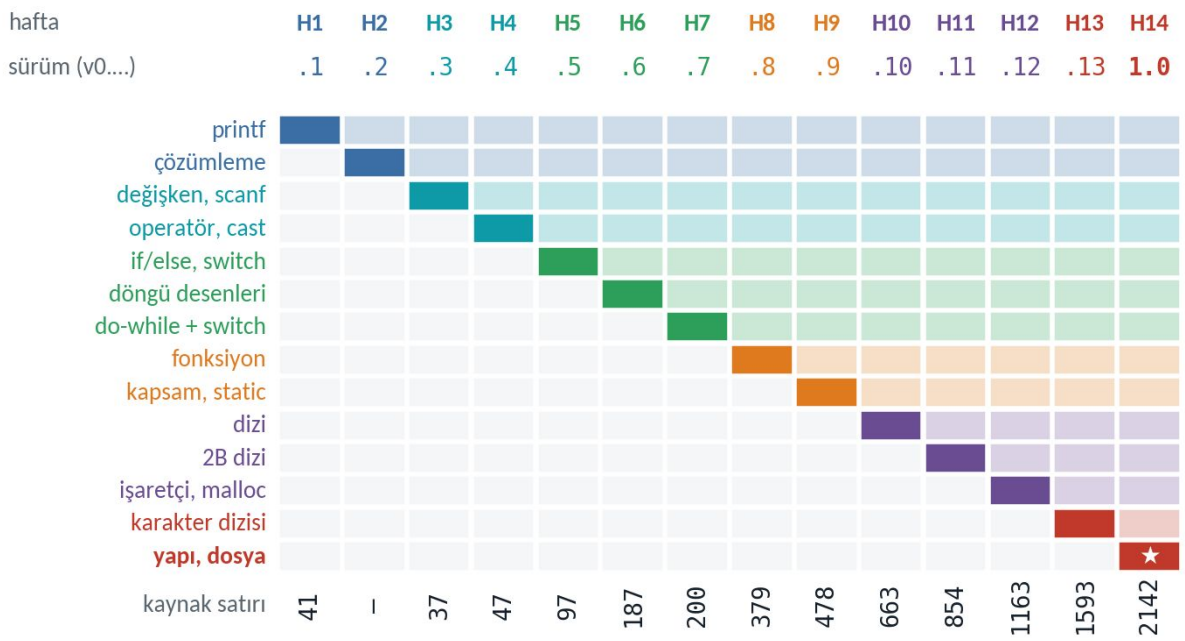
Y2K'nın bir benzeri zaman damgalarında bekler. POSIX sistemleri zamanı 1970-01-01 00:00:00 UTC'den beri geçen saniye olarak tutar (C standardı `time_t`'nin gösterimini uygulamaya bırakır). Bu sayı işaretli 32 bitlik bir tamsayıda tutulursa en büyük değeri $2^{31} - 1 = 2\,147\,483\,647$'dir: `H14_K14_y2k_2038.c` bunu "2038-01-19 03:14:07 UTC" olarak yazdı; bir saniye sonrası 32 bitte en küçük değere taşar ve "1901-12-13 20:45:52 UTC" olarak okunur. Bu sistemde `time_t` 8 bayttır, sorun yoktur; ama zamanı dosyalarda ya da ağ protokollerinde 32 bitlik alanlarda saklayan kayıt biçimleri (Y2K'daki iki haneli yıl gibi) etkilenir. Senato komitesinin raporu da 2038'i bazı UNIX sistemlerindeki zamanlayıcıların taşacağı bir tarih olarak anar (United States Senate Special Committee on the Year 2000 Technology Problem, 2000).

11. Özet, Kavram Haritası ve Terim Sözlüğü

- **Yapı** ilişkili verileri tek bir kayıt tipinde toplar; `.` ile üyeye, `->` ile işaretçinin gösterdiği yapının üyesine erişilir. Atama bütün üyeleri (diziler dahil) kopyalar; karşılaştırma alan alan yapılır.
- Derleyici üyeleri hizalamak için **dolgu** ekler: `sizeof` üyelerin toplamından büyük olabilir; üye sırası boyutu değiştirir.
- Dosyaya bir `FILE *` akışıyla erişilir; kip doğru seçilir, `fopen`/`fclose` ve okuma dönüşleri denetlenir; veri tampondan ancak `fflush`, `fclose` ya da normal çıkışla işletim sistemine gider.
- Metin (CSV) taşınabilir ve okunur; ikili dosya hızlı ve kayıpsız ama taşınabilir değildir. Dosyadan gelen veri güvenilmezdir.

> NOT · **Büyük Sorunun yanıtı**
>
> Program kapandığında RAM'deki her şey (yığındaki değişkenler, öbekteki bloklar, C kütüphanesinin tamponları) yok olur; kalıcı olan yalnızca **diske yazılmış** baytlardır. Bir "kayıt" bellekte bir **yapıdır**: üyeleri bildirim sırasıyla yan yana durur, gerekirse araya hizalama dolgusu girer (Mini-OBS'nin 144 baytlık `Ogrenci`'sinde dolgu yok). Diskte iki biçimi olabilir: insanın ve her makinenin okuyabildiği **metin** (CSV satırı) ya da bellekteki baytların birebir kopyası olan **ikili** kayıt (16 baytlık başlık + 144 bayt). Veri, ancak `fflush`/`fclose` ya da normal çıkışla işletim sistemine teslim edilir; HA4'te teslim edilmeyen üç kayıt kayboldu. Y2K'nın gösterdiği gibi, kaydın alanlarını ve biçimini seçmek programdan uzun yaşayan bir karardır.

Şekil 14.10
Dönem ve kavram haritası: her satır bir haftanın kavramı; koyu hücre kavramın Mini-OBS'ye girdiği sürüm, açık hücreler sonraki sürümlerde de kullanıldığını gösterir; alt satır sürümün .c + .h satır sayısı (gerçek dosyalardan)



Terim sözlüğü

Tablo 14.9
Hafta 14 terimleri (Bölüm 6.8 standardıyla)

Türkçe	İngilizce	Kısa tanım
yapı / üye	structure / member	farklı tipte üyeleri tek ad altında toplayan tip
belirlenmiş ilk değer	designated initializer	`{.ad = …}`: üyeye adıyla ilk değer (C99)
numaralandırma / birleşim	enumeration / union	adlı int sabitleri / üyeleri aynı adreste başlayan tip
hizalama / dolgu	alignment / padding	adres katı kuralı / üyeler arasına eklenen boş baytlar
dosya akışı	file stream	dosyaya `FILE *` üzerinden erişim; konum, gösterge ve tampon
tampon / boşaltma	buffer / flush	ara bellek alanı / içeriğini işletim sistemine iletme
metin dosyası / ikili dosya	text file / binary file	karakter olarak / bellekteki baytlarıyla saklanan dosya
dosya sonu göstergesi	end-of-file indicator	okuma dosya sonuna çarpınca kurulur (`feof`)
onaltılı döküm	hex dump	dosya baytlarının onaltılı gösterimi (`od`, Format-Hex)

12. Kaynaklar

Bölümler için ayrıca: yapılar, birleşimler ve numaralandırmalar King (2008, Bölüm 16) ve Deitel ve Deitel (2022, Bölüm 10); dosyalar ve akışlar King (2008, Bölüm 22), Deitel ve Deitel (2022, Bölüm 11) ve Seacord (2013, Bölüm 8).

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Microsoft. (2022). *rename, _wrename*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/rename-wrename

Microsoft. (2025). *fopen, _wfopen*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/fopen-wfopen

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.

United States Senate Special Committee on the Year 2000 Technology Problem. (2000). *Y2K aftermath – crisis averted: Final committee report* (S. Prt. 106-42). U.S. Government Printing Office. https://www.govinfo.gov/app/details/GOVPUB-Y1_3-PURL-LPS90964

Willemssen, J. C. (1997). *Year 2000 computing crisis: Success depends upon strong management and structured approach* (GAO/T-AIMD-97-173) [Testimony]. U.S. General Accounting Office. https://www.govinfo.gov/content/pkg/GAOREPORTS-T-AIMD-97-173/pdf/GAOREPORTS-T-AIMD-97-173.pdf